
3d Computer Vision

§4 Data Representation

Prof. Dr. Georg Umlauf

Content

1. Overview

- a. Intro: The 3d reconstruction pipeline
- b. Topology vs. geometry

2. Point cloud data structures

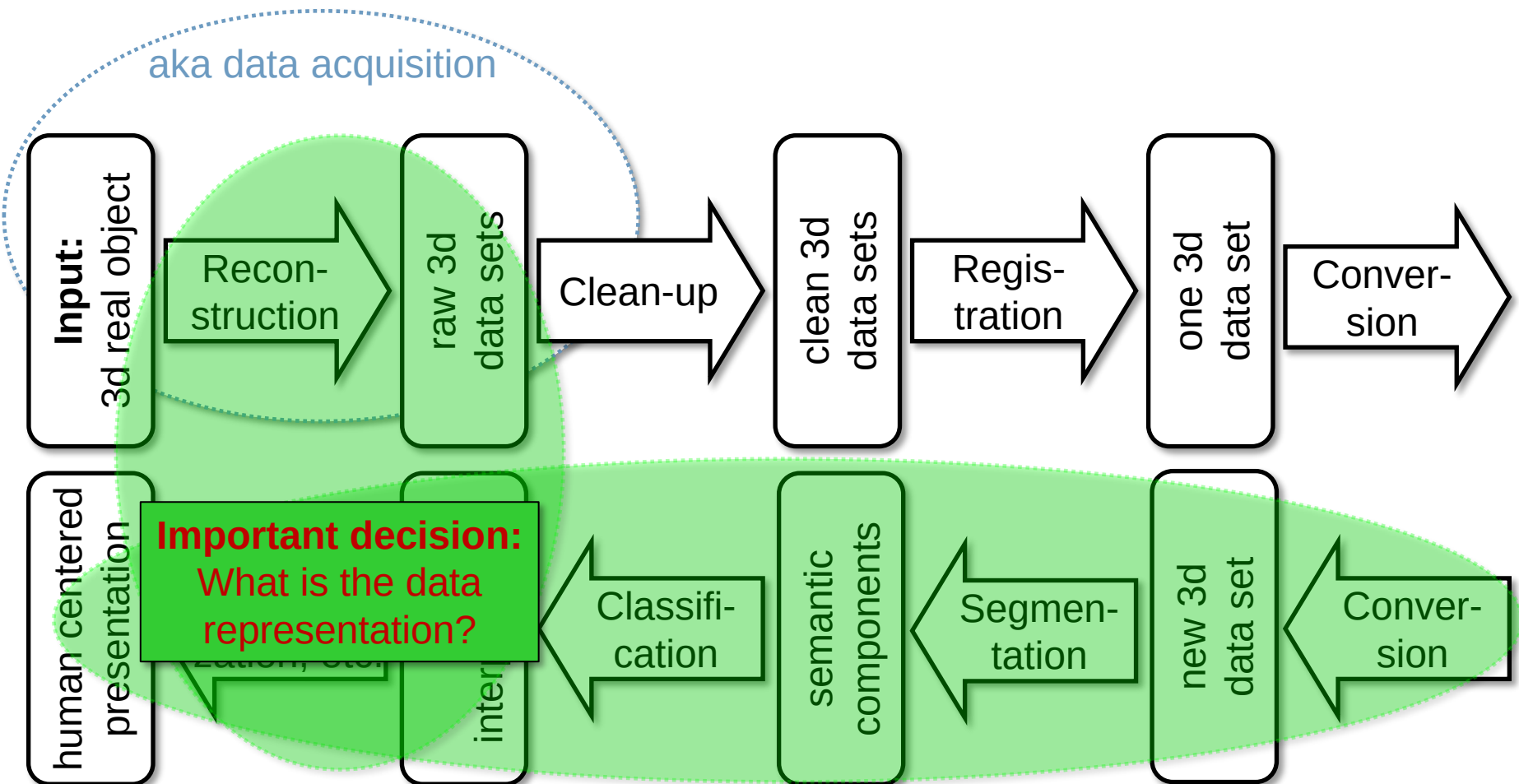
- a. Grids
- b. Quadtrees, octrees
- c. *kd*-trees
- d. BSP-trees

3. Mesh data structures

- a. Mesh basics
- b. Half-edge data structure

1. Overview

3d Reconstruction Pipeline



Content

1. Overview

- a. Intro: The 3d reconstruction pipeline ✓
- b. **Topology vs. geometry**

2. Point cloud data structures

- a. Grids
- b. Quadtrees, octrees
- c. kd-trees
- d. BSP-trees

3. Mesh data structures

- a. Mesh basics
- b. Half-edge data structure

1. Overview

Topology vs. Geometry (1)

Topology:

Set of properties of an object, that do **not** change under rigid body transformations.

► The structure of the model.

In the example: The polygon has three vertices, which are adjacent via edges.

Geometry:

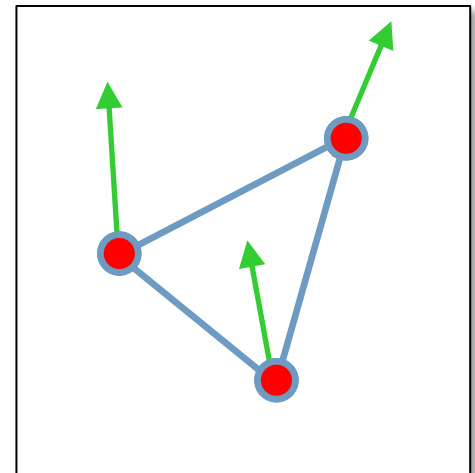
The „instantiation“ of the topology by specification of its spatial position (aka coordinates).

► The shape of the model.

In the example: The coordinates of the corners.

Attributes:

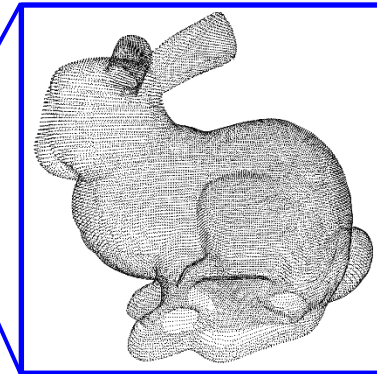
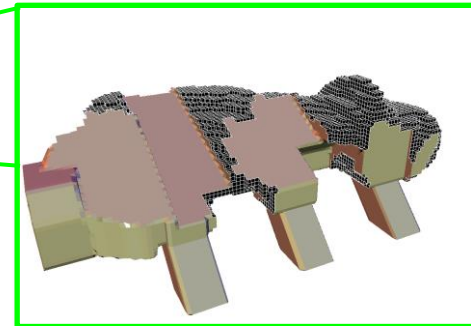
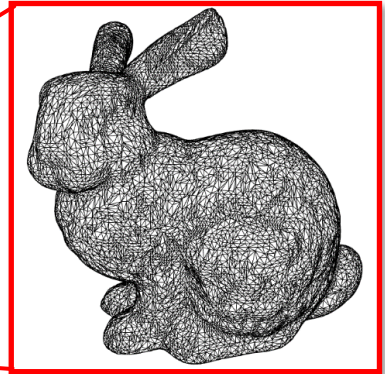
E.g., color, normal, texture, transparency, material properties, etc.



1. Overview

Topology vs. Geometry (2)

- Topology and geometry can be represented/stored explicitly or implicitly.
- **Explicit representation of topology**, e.g., point-meshes, etc.
- **Implicit representation of topology**, e.g., gridded data, space partitioning, etc.
- **Explicit representation of geometry**, e.g., point cloud, CAD-model, etc.
- **Implicit representation of geometry**, e.g., scalar field, zero-set of a given function, etc.



1. Overview

Topology vs. Geometry (3)

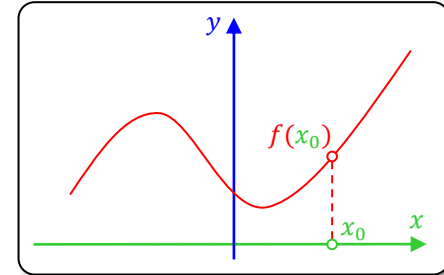
- **Remark:** Implicit geometry often comes with implicit topology (gridded), because of a method called **marching cubes**, which is
 - simple and easy to implement and
 - relatively robust.
- **Remark:** Sometimes geometry and topology is implicitly defined in the attributes.
 - Example: Iso-lines in a pressure field, aka isobars.
 - Example: Iso-lines in a temperature field, aka isothermal lines.
 - Example: Iso-lines in a height-field, aka contour lines.
 - Example: Line of largest gradient value in an RGB-image, aka contour or silhouette
 - Example: Iso-surface in stack of MRI-grey-value images.

1. Overview

Dimensionality of the Geometry (1)

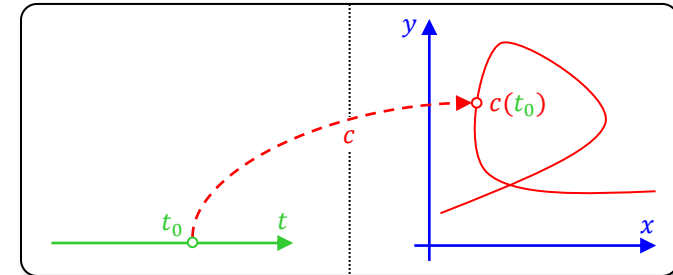
1) 1.5-dimensional, aka function: 1d-functions

$$f: \mathbb{R} \rightarrow \mathbb{R}, x \mapsto f(x) = y$$



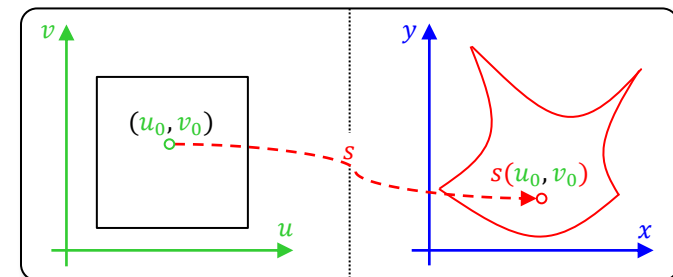
2a) 2-dimensional, aka curve: 1-parametric curves, aka 1-manifolds embedded in $\mathbb{R}^2$

$$c: \mathbb{R} \rightarrow \mathbb{R}^2, t \mapsto c(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$$



2b) 2-dimensional: 2-parametric surfaces embedded in $\mathbb{R}^2$

$$s: \mathbb{R}^2 \rightarrow \mathbb{R}^2, (u, v) \mapsto s(u, v) = \begin{pmatrix} x(u, v) \\ y(u, v) \end{pmatrix}$$



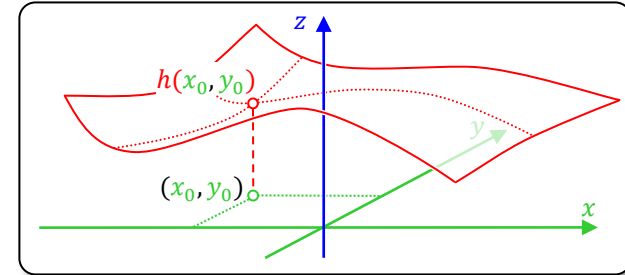
- Example: Re-meshing of the physical domain for simulations.

1. Overview

Dimensionality of the Geometry (2)

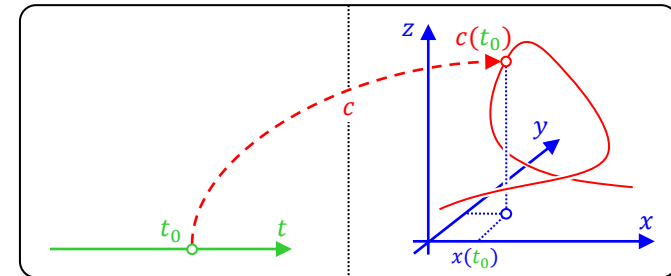
2c) 2.5-dimensional, aka height-field: 2d-functions, height fields, 2d-scalar fields

$$h: \mathbb{R}^2 \rightarrow \mathbb{R}, (x, y) \mapsto h(x, y) = z$$



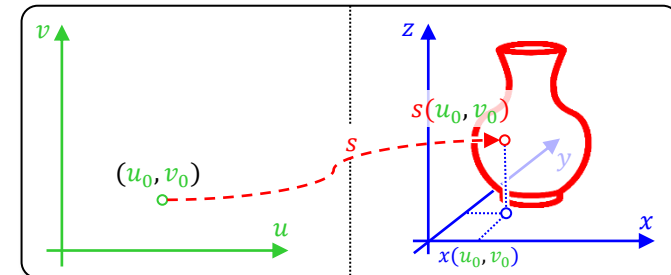
3a) 3-dimensional, aka 3d-curve: space curve, 1-manifolds embedded in $\mathbb{R}^3$

$$c: \mathbb{R} \rightarrow \mathbb{R}^3, t \mapsto c(t) = \begin{pmatrix} x(t) \\ y(t) \\ z(t) \end{pmatrix}$$



3b) 3-dimensional, aka surface: 2-parametric surfaces, 2-manifolds embedded in $\mathbb{R}^3$

$$s: \mathbb{R}^2 \rightarrow \mathbb{R}^3, (u, v) \mapsto s(u, v) = \begin{pmatrix} x(u, v) \\ y(u, v) \\ z(u, v) \end{pmatrix}$$



1. Overview

Dimensionality of the Geometry (3)

3c) 3-dimensional aka 3d-scalar-field: scalar-values embedded in $\mathbb{R}^3$

$$S: \mathbb{R}^3 \rightarrow \mathbb{R}, (u, v, w) \mapsto S(u, v, w) = s$$

- Example: CT-, MRI-, ultra-sound-data and extract iso-surfaces.

3d) 3-dimensional: k d-vector-field embedded in $\mathbb{R}^3$

$$V: \mathbb{R}^3 \rightarrow \mathbb{R}^k, (u, v, w) \mapsto V(u, v, w) = \begin{pmatrix} x_1(u, v, w) \\ \vdots \\ x_k(u, v, w) \end{pmatrix}$$

- For example: FEM-simulations, wind tunnel, etc.

3e) 3-dimensional: tensor-field embedded in $\mathbb{R}^3$...

- Example: FEM-simulations.

1. Overview

Conversion

- Data acquisition often yields point clouds, i.e., a discrete representation.
- **But:** Often a continuous representation is better for post-processing.
- ▶ **Conversion** from point-cloud to continuous representation:
 - **(Scattered data) interpolation/approximation:** 1.5d-point-cloud to function or polygon
 - **Curve interpolation/approximation:** 2d-point-sequence to parametric curve or polygon
 - **Scattered data interpolation/approximation:** 2.5d-point-cloud to 2d-function
 - **Surface reconstruction:** 3d-point-cloud to parametric surface
 - **(Point cloud) triangulation:** 3d-point-cloud to mesh

Content

1. Overview

- a. Intro: The 3d reconstruction pipeline 
- b. Topology vs. geometry 

2. Point cloud data structures

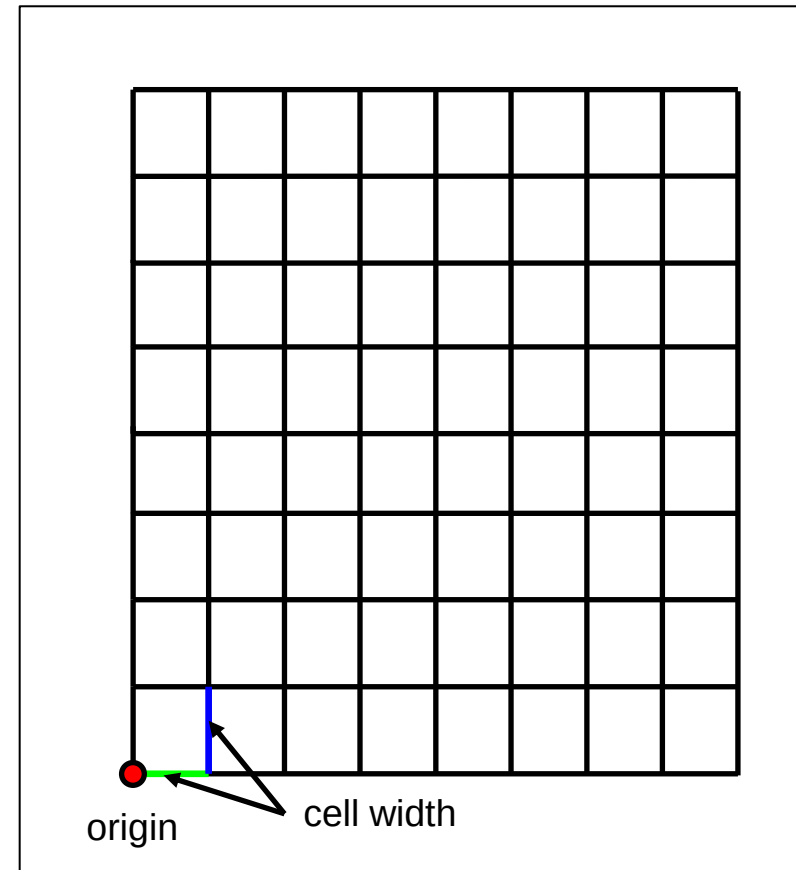
- a. **Grids**
- b. Quadtrees, octrees
- c. *kd*-trees
- d. BSP-trees

3. Mesh data structures

- a. Mesh basics
- b. Half-edge data structure

2a. Gridded data

- Orthogonal, uniform grid.
- Topology is determined by number of cells in x - and y -direction (and z -direction).
- Geometry is determined by
 - origin and cell width or
 - values/attributes in the cell.
- The cells are called **pixel** or **voxel**.
- **Example:** Origin = $(-1, -1)$,
 $x_{\text{dim}} = 0.2$, $y_{\text{dim}} = 0.25$,
 $n_x = 8$, $n_y = 8$
- **Example:** RGB-image



Content

- 1. Overview ✓
 - a. Intro: The 3d reconstruction pipeline ✓
 - b. Topology vs. geometry ✓
- 2. Point cloud data structures
 - a. Grids ✓
 - b. Quadtrees, octrees
 - c. kd-trees
 - d. BSP-trees
- 3. Mesh data structures
 - a. Mesh basics
 - b. Half-edge data structure

2b. Quadrees

- In many applications it is important to store 2d/3d point sets with **fast access** and with as **little memory** as possible.
- One option is based on so-called **Quadrees** in 2d and **Octrees** in 3d.
 - Quad-/Octrees support a fast search for leaves in the tree, that represent objects/points in the scene.
- Applications of quadrees/octrees:
 - face indexing (e.g., in GIS programs), range queries,
 - collision detection in 2d/3d,
 - hidden surface removal in terrain data,
 - ray-casting, ray-tracing, etc.

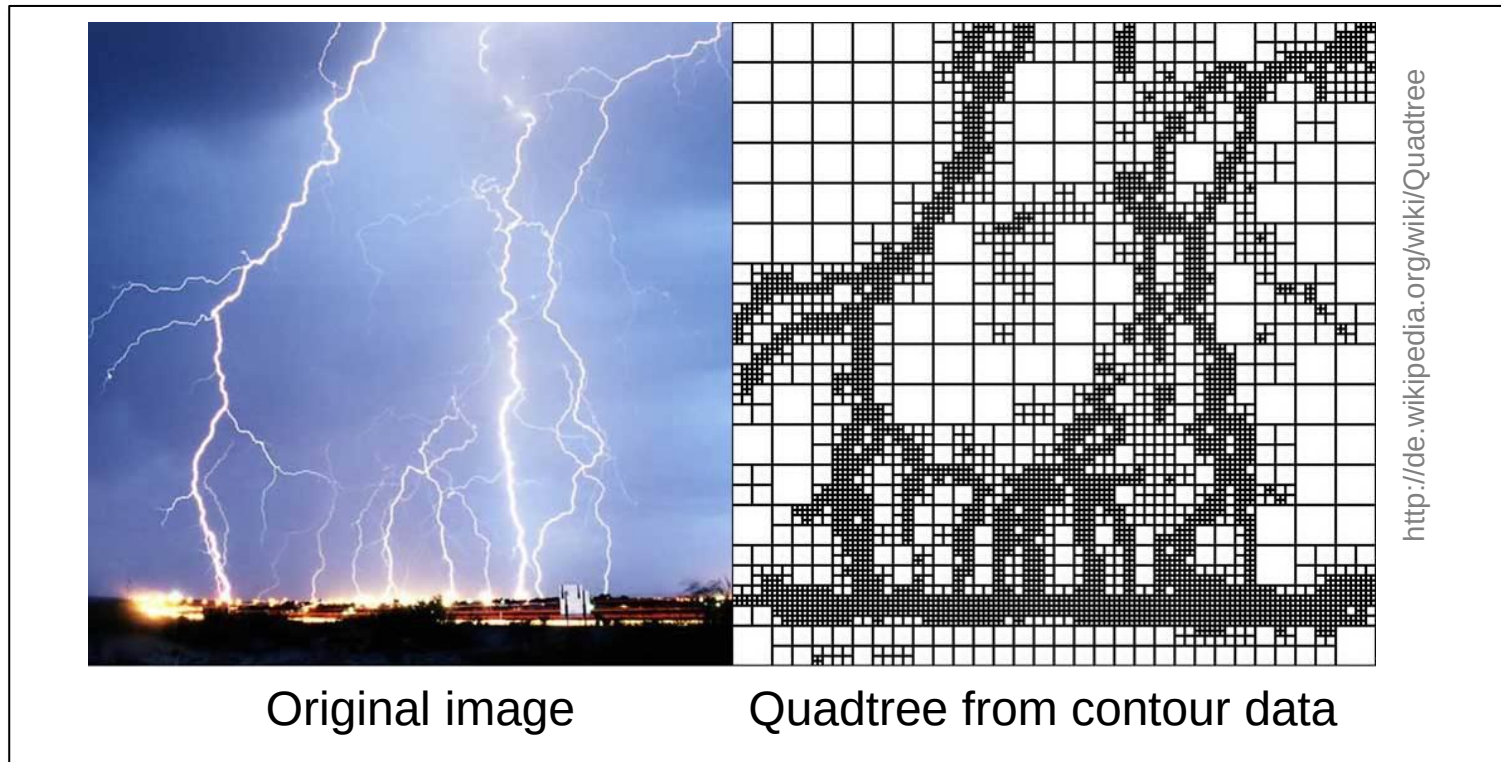
2b. Quadtrees

Quad-tree approach (grid-method) for range-queries

- Range Query: Determine points that are inside a user defined axis aligned query range.
- Place regular grid over the set of points P .
- Consider only points in grid cells, that intersect the query range.
- **Efficient**, if the points are *uniformly* distributed.
- **Inefficient**, if the points are concentrated in one grid area and the grid has many empty grid cells.

2b. Quadrees

■ Example



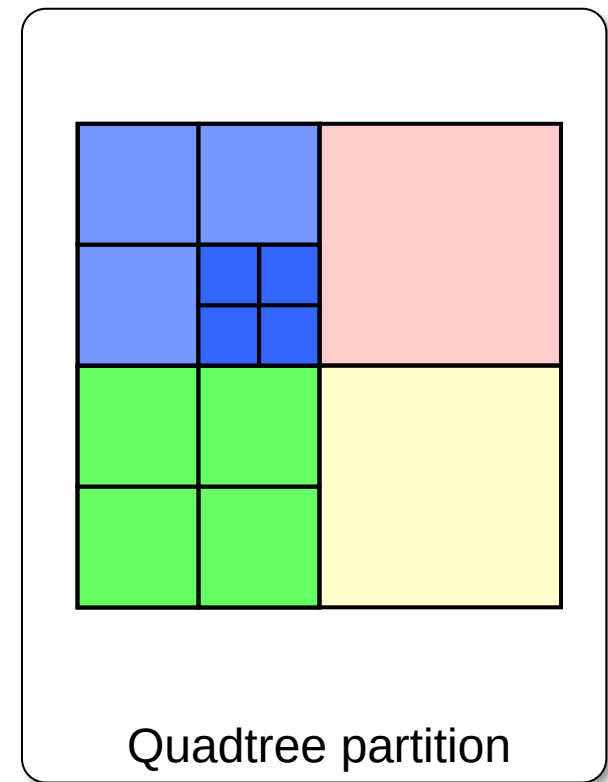
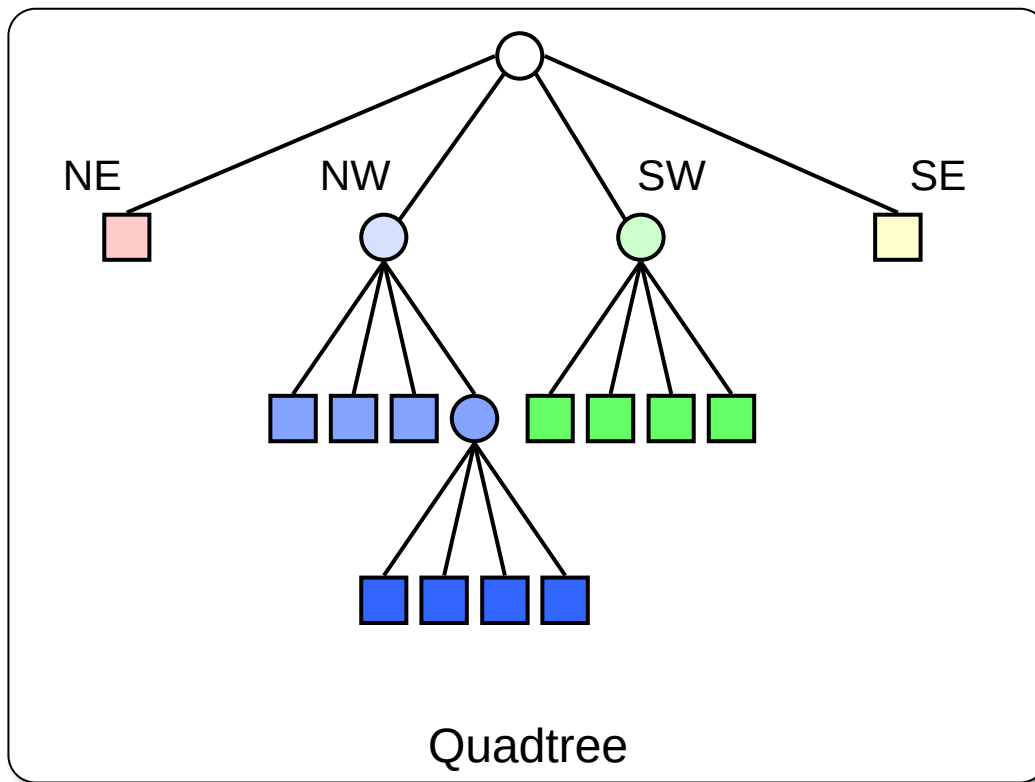
- Quadtrees can store different types of data.
 - We focus here on point clouds in the plane!

2b. Quadrees

- A *quadtree* is a tree where every inner node has four child nodes:
 - Nodes correspond to square regions.
 - If a node v has children, these correspond to the four sub-squares (quadrants) of the square corresponding to v .
 - The children of the roots of sub-trees are named by the quadrants:
 - NE – North-east-quadrant,
 - NW – North-west-quadrant ,
 - SW – South-west-quadrant,
 - SE – South-east-quadrant.
- ▶ The squares of all leaves partition the square corresponding to the root, this is the so-called *quadtree-partition*.

2b. Quadrees

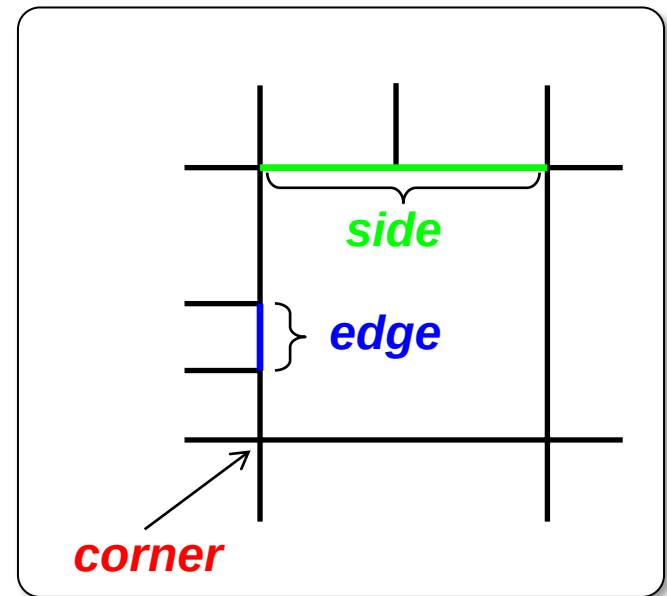
- An example of a quadtree and the corresponding partition.



2b. Quadrees

Terminology

- The four knots of the corners of the squares are called *corners*.
- Line segments, connecting corners of the same square are called *sides of the square*.
- Edges of the quadtree-partition, contained in the sides of a square, are called *edges of the square*.
 - ▶ A side contains at least one edge.
- Two squares are *neighbors*, if they have a common edge.



2b. Quadrees

Formal Definition

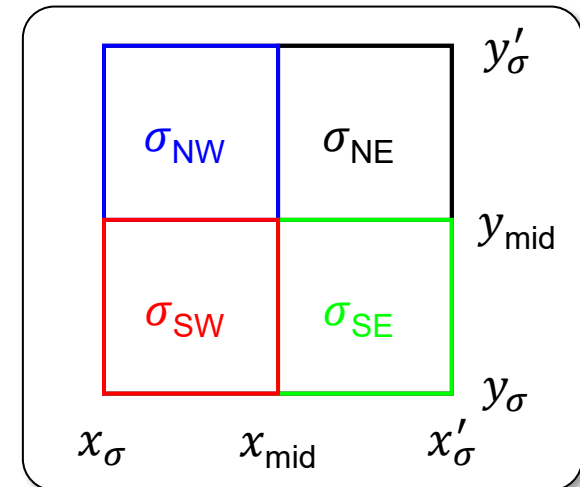
- Let $\sigma := [x_\sigma, x'_\sigma] \times [y_\sigma, y'_\sigma]$ be a square and P a set of points in $\mathbb{R}^2$.
- If $\text{card}(P) \leq 1$, the quadtree is a single leaf, that stores σ and P .
- Otherwise denote
 - by σ_{NE} , σ_{NW} , σ_{SE} and σ_{SW} the four quadrants of σ
 - with $x_{\text{mid}} := (x_\sigma + x'_\sigma)/2$ and $y_{\text{mid}} := (y_\sigma + y'_\sigma)/2$ and

$$P_{\text{NE}} := \{p \in P : p_x > x_{\text{mid}} \text{ and } p_y > y_{\text{mid}}\}$$

$$P_{\text{NW}} := \{p \in P : p_x \leq x_{\text{mid}} \text{ and } p_y > y_{\text{mid}}\}$$

$$P_{\text{SW}} := \{p \in P : p_x \leq x_{\text{mid}} \text{ and } p_y \leq y_{\text{mid}}\}$$

$$P_{\text{SE}} := \{p \in P : p_x > x_{\text{mid}} \text{ and } p_y \leq y_{\text{mid}}\}$$



2b. Quadrees

- The quadtree has a root v , storing the square σ , which can be accessed by $\sigma(v)$.
- Each inner node v can have four children:
 - The NE-child is the root of the quadtree for the set P_{NE} of points contained in the square σ_{NE} .
 - The NW-child is the root of the quadtree for the set P_{NW} of points contained in the square σ_{NW} .
 - The SW-child is the root of the quadtree for the set P_{SW} of points contained in the square σ_{SW} .
 - The SE-child is the root of the quadtree for the set P_{SE} of points contained in the square σ_{SE} .

2b. Quadrees

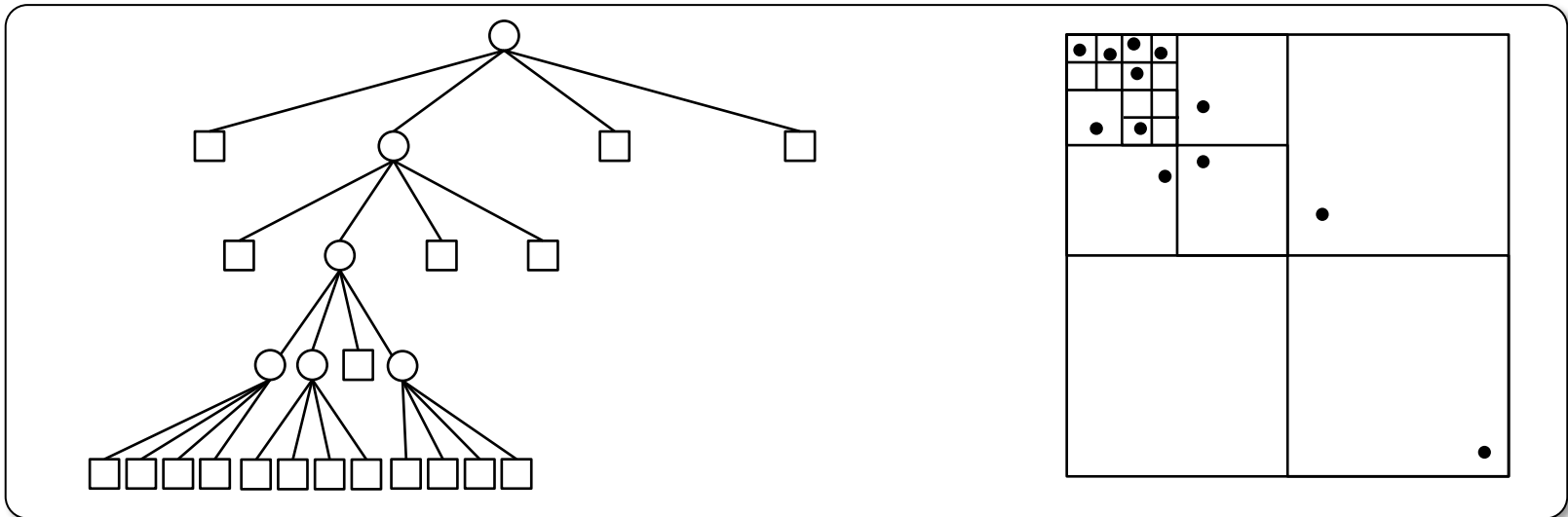
- This recursive definition gives immediately an algorithm to construct a quadtree:
 - Start with an initial square containing all points (aka bounding box).
 - Split the current square into four quadrants and partition the points accordingly.
 - Construct recursively the quadtrees for the four quadrants with the corresponding partitionings.
 - Stop, when the point sets contain less than two points.

- If the initial square is not known a priori, the smallest enclosing square (aka bounding box) of the point set can be computed in linear time.

2b. Quadrees

Observation

- After each step, the squares are split into four smaller squares.
 - This is also applied to the associated point sets.



- ▶ Quadtrees can be badly *unbalanced*.
- ▶ Its depth and size depend on the distance of the points and the size of the initial square!

2b. Quadrees

Lemma

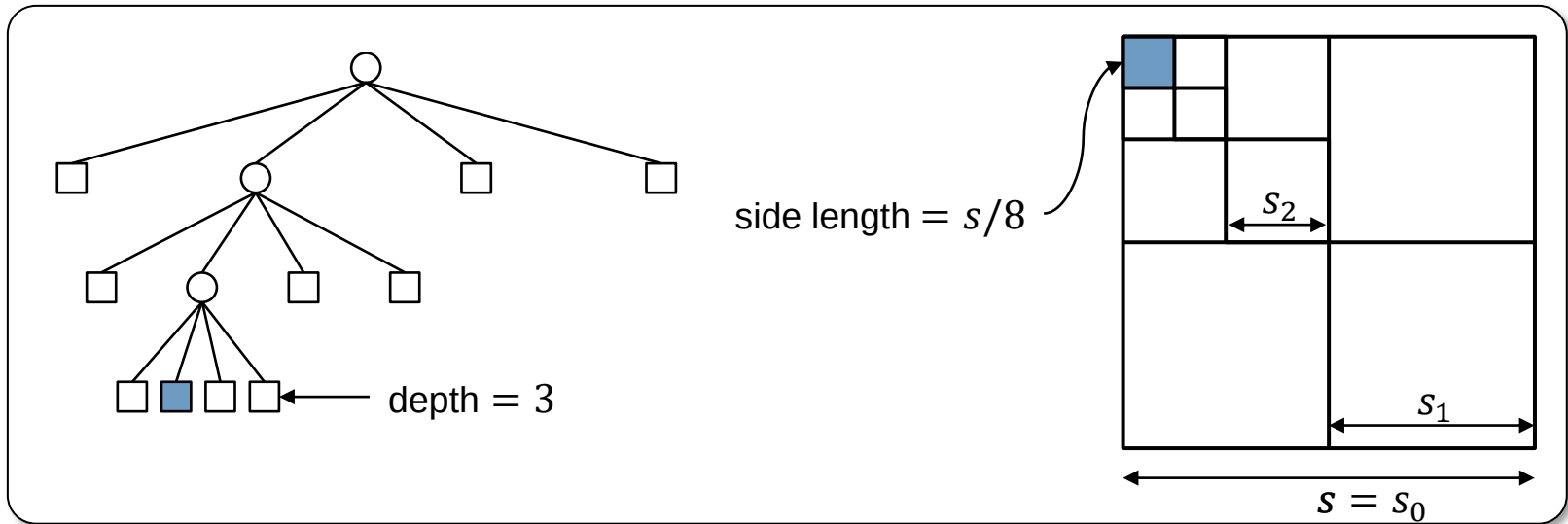
The depth of a quadtree for a point set P of points in the plane is at most, $\log_2 \left(\frac{s}{c} \right) + \frac{3}{2}$ where

- c is the smallest distance between two points in P and
- s is the length of the sides of the bounding box of P .

Proof:

...

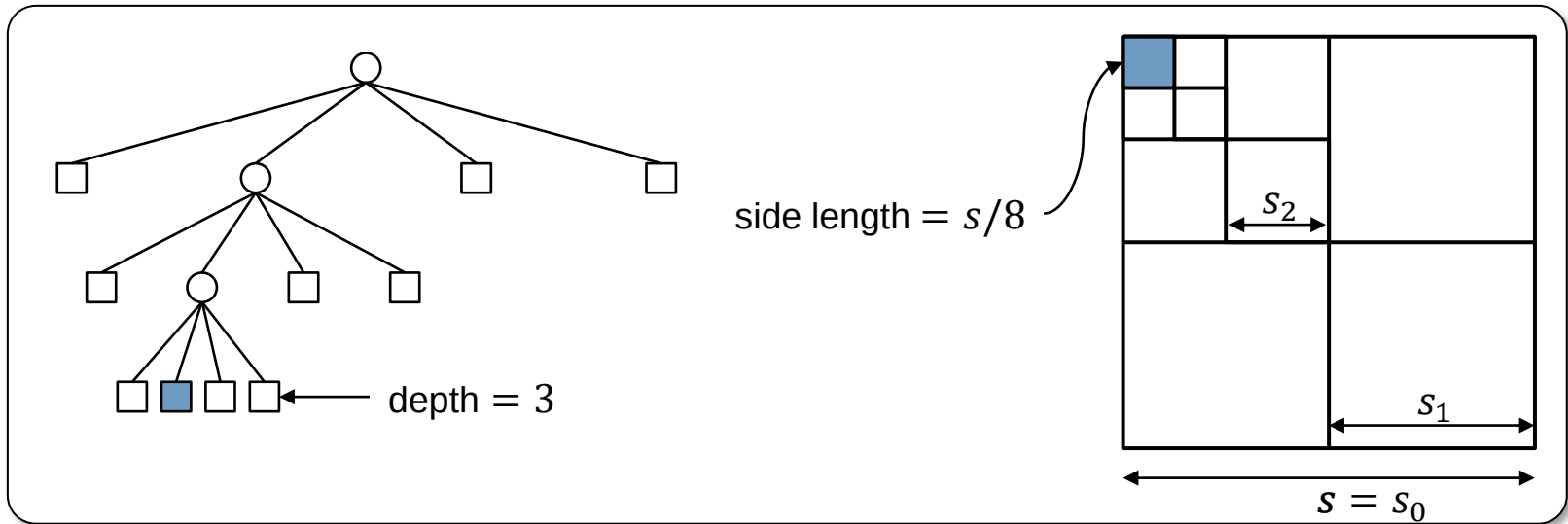
2b. Quadrees



Proof:

- The size of the squares is halved on successive levels of the tree.
- ▶ For a node of depth i the side length is $s_i = s/2^i$. The root has depth 0.
- The largest distance of two points in a square is the length of the diagonal, which is for depth i given by $s_i\sqrt{2} = s\sqrt{2}/2^i$.

2b. Quadrees



- An internal node of depth i has at least two points in its corresponding square, i.e., its side length s_i satisfies $s_i\sqrt{2} = s\sqrt{2}/2^i \geq c$.
- Solving for i yields

$$i \leq \log_2 \left(\frac{s\sqrt{2}}{c} \right) = \log_2 \frac{s}{c} + \frac{1}{2}.$$

- The depth of a quadtree is the maximal depth of an internal node plus one.



2b. Quadrees

Proposition

A quadtree of depth d that stores a set of n points

- has $O((d + 1) n)$ nodes and
- can be constructed on $O((d + 1) n)$.

Proof:

1. Number of nodes:

- Every inner node has four children.
- ▶ The total number of leaves is one plus three times the number of inner nodes.
- ▶ It suffice to estimate the number of inner nodes.

2b. Quadrees

- An internal node has at least one point in its corresponding square and squares of nodes on the same level are disjoint and cover exactly the initial square.
- ▶ The total number of internal nodes on an arbitrary level is at most n .
- ▶ If the root has level 0, the claim for the number of nodes follows as $\#inner\ node + \#leaves \leq n \cdot (d + 1) + 3 \cdot n \cdot (d + 1) + 1 = O((d + 1)n)$.

2. Construction time:

- The most time-consuming operation in every step of the recursive algorithm is to partition the points of a square to its four quadrants.
- This time is spent for every internal node and is linear in the number of points contained in the actual square.
- Because the total number of points associated to internal nodes at an arbitrary level is at most n , the time bound is $O((d + 1)n)$.

2b. Quadtrees*

- An important operation on quadtrees is *Finding Neighbors*.

Input: A node v and a direction δ

$\delta \in \{\text{North, South, West, East}\}.$

Output: A node v' , where $\sigma(v')$ is a neighbor of $\sigma(v)$ in direction δ .

- Often input and output nodes are leaves:
 - This corresponds to the search for adjacent squares in the quadtree-partition.

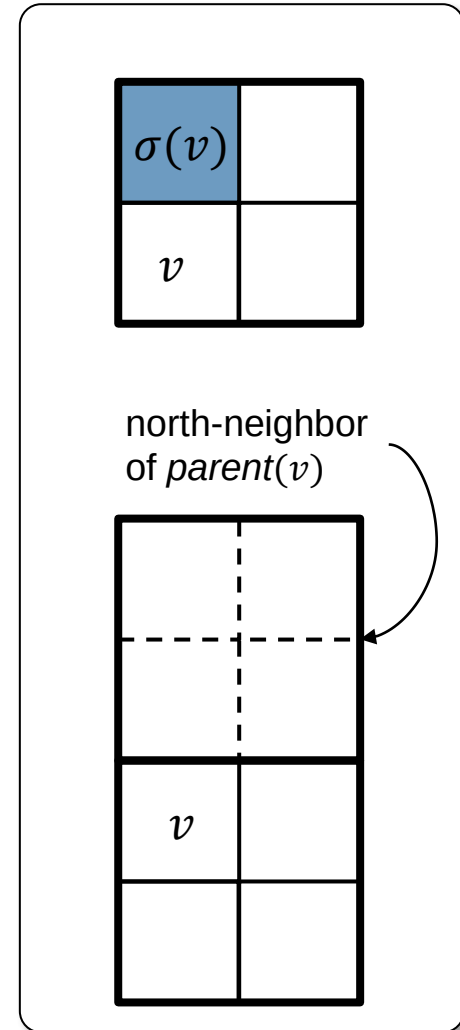
2b. Quadtrees*

- Basic idea
 - The input node v can also be an internal node!
 - The algorithm searches a node v' with:
 - $\sigma(v')$ is adjacent to $\sigma(v)$ in direction δ **AND**
 - both nodes have the same depth.
 - If there is no such node, compute the deepest node
 - whose depth is at most the depth of v **AND**
 - whose square is adjacent in direction δ .
 - If there is also no such node in direction δ , return NULL.

2b. Quadtrees*

■ Example

- Assume we search for the north neighbor of v .
- If v is a SE- or SW-child, the north neighbor of v is its NE- or NW-sister node.
- If v is a NE- or NW-child, search recursively the north neighbor w of its parent node.
 - If w is an internal node, the north neighbor of v is a child of w .
 - If w is a leaf, the north neighbor of v is w .



2b. Quadrees*

```
1 Node FindNorthNeighbor(Node v, QuadTree T)
2 {
3     if (v == root(T)) then return null;
4     p = parent(v);
5     if (v == SW-child(p)) then return NW-child(p);
6     if (v == SE-child(p)) then return NE-child(p);
7     w = FindNorthNeighbor(p,T);
8     if (w == null or w is a leaf) then return w;
9     else
10         if (v == NW-child(p)) then return SW-child(w);
11         else return SE-child(w);
12 }
```

2b. Quadtrees*

- The algorithm takes per recursive call only $O(1)$ time and the depth of the actual node v is decreased by one.
- ▶ Runtime is linear in the depth of the quadtree.

Proposition

Let T be a quadtree of depth d . The neighbor of a node v in T in each direction δ can be computed in $O(d + 1)$.

2b. Quadrees

■ **Advantages of quadrees and octrees**

- Simple to implement.
- Efficient algorithms, if well balanced.
- Can handle data online.

■ **Disadvantages of quadrees and octrees**

- Can get badly unbalanced, dependent on distribution of data.
- Inefficient algorithms, if badly balanced.

■ **Alternatives**

- *kd*-trees, range-trees, bsp-trees, trapezoidal maps, half-edges, etc.
- Dependent on application...

Content

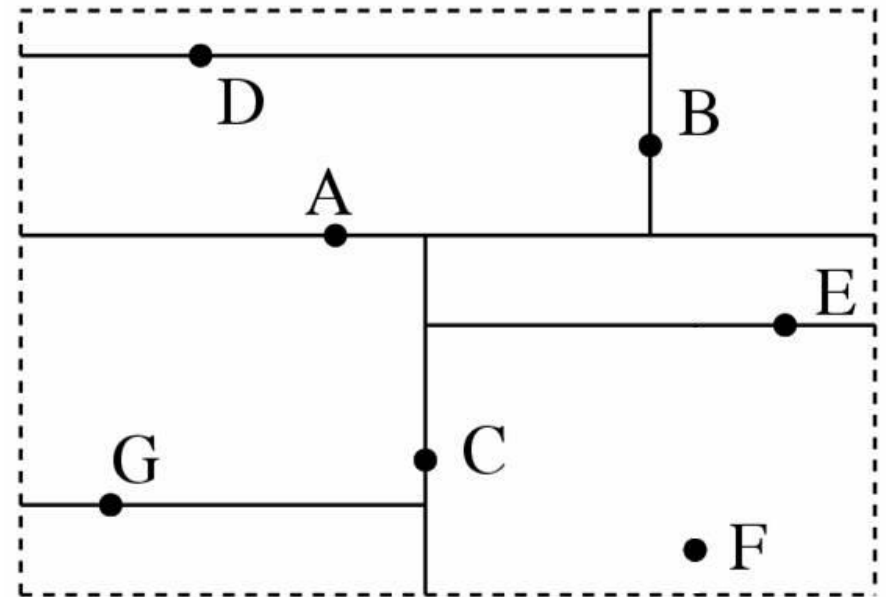
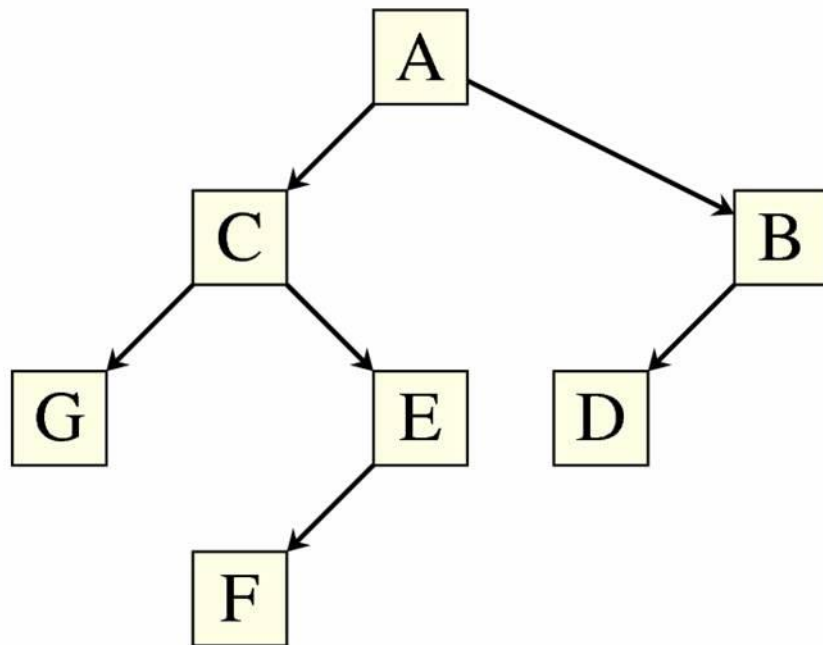
- 1. Overview ✓
 - a. Intro: The 3d reconstruction pipeline ✓
 - b. Topology vs. geometry ✓
- 2. Point cloud data structures
 - a. Grids ✓
 - b. Quadtrees, octrees ✓
 - c. **kd-trees**
 - d. BSP-trees
- 3. Mesh data structures
 - a. Mesh basics
 - b. Half-edge data structure

2c. kd-trees

- In the sequel only two-dimensional data, i.e., $k = 2$.
- Extension to higher dimensions is then straight-forward.
- Idea
 - Partition 2d search space like a binary search tree for one-dimensional search spaces.
 - Use alternating the x - und y -coordinates as keys.
 - Construct a binary tree, representing a partition of the plane:
 - The knots correspond to the n points in 2d.
 - On an even level of the tree (level=depth+1) use the x -coordinate as key, otherwise the y -coordinate.

2c. kd-trees

- A 2d-search tree and the resulting partition of the plane.



2c. kd-trees

Properties

- For a knot in the tree v_x representing point p_x , whose key is its x -coordinate (even level), we have:
 - The left sub-tree of v_x contains the points left of p_x .
 - The right sub-tree of v_x contains the points right of p_x .
 - The sub-trees of children of v_x are determined by y -coordinates.
- For a knot in the tree v_y representing point p_y , whose key is its y -coordinate (odd level), we have:
 - The left sub-tree of v_y contains the points below of p_y .
 - The right sub-tree of v_y contains the points above of p_y .
 - The sub-trees of children of v_y are determined by x -coordinates.

2c. kd-trees

Run time

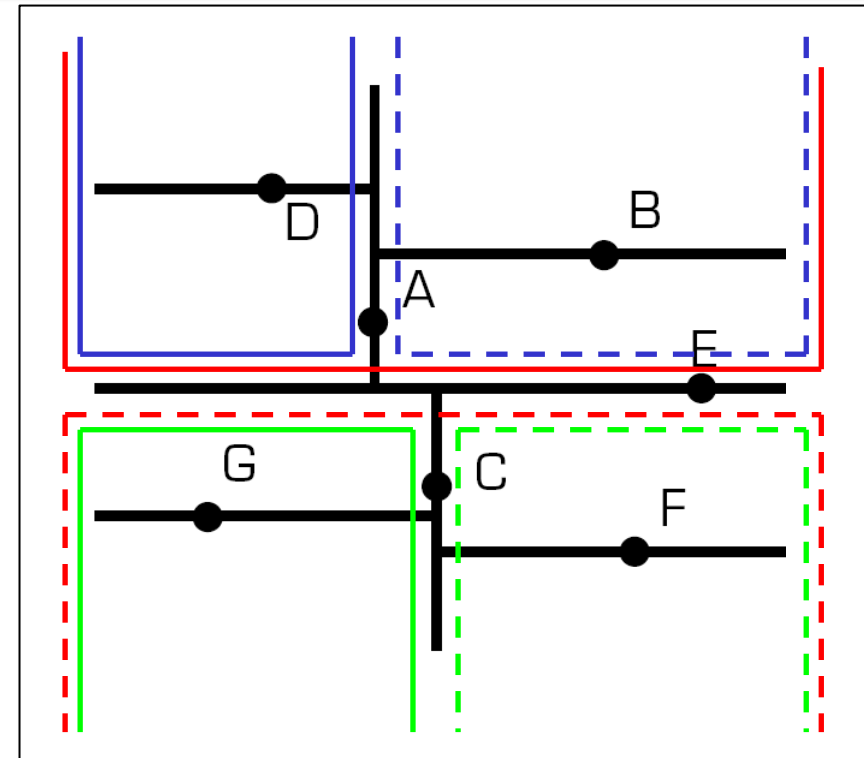
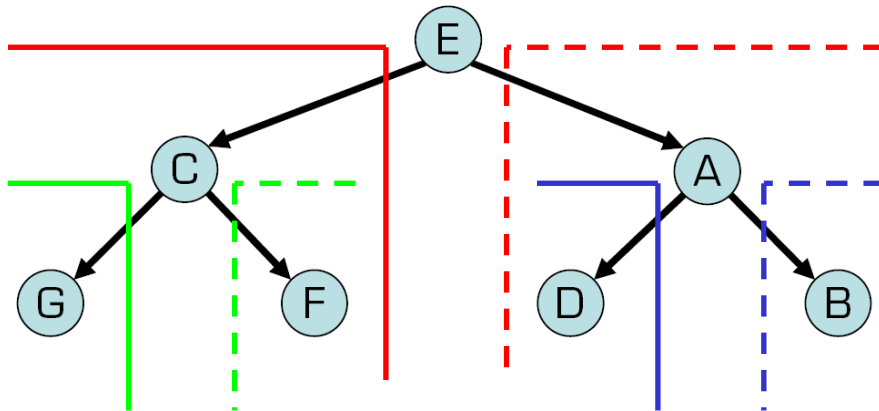
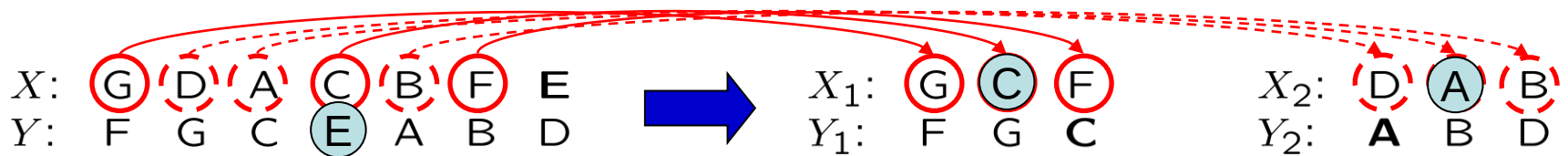
- If the 2d-tree is balanced, the height of the tree is $O(\log a)$ since it is a binary tree.
- If the 2d-tree is unbalanced, a path in the tree might degenerate to a linear list resulting in a linear run time.
- Goal: The construction of the tree must guarantee a good balance.
- Solution: Partition the points at the **median** of the sequence of x - and y -coordinates.

Partitioning at the median

- Sort all points both by x - and y -coordinates.
 - ▶ Two sorted sequences X and Y .
- Subdivide Y at its median and take it as the root of the tree.
 - ▶ Two sub-sequences Y_1 and Y_2 .
- Partition X into two sequences X_1 and X_2 , such that X_1 contains the same points as Y_1 and X_2 the same points as Y_2 .
- Partition X_1 and X_2 recursively at the median and subdivide Y_1 and Y_2 accordingly as above, until these sequences contain only one point.
 - These points are the leafs of the tree.

2c. kd-trees

■ Example for the partition



2c. kd-trees

- Global, pre-sorted sequences **X** and **Y**.

ConstructBalanced2DTree

Input: left index l , right index r , knot k , direction d

```
1: if ( $l \leq r$ ) then {
2:    $m = \left\lfloor \frac{l+r}{2} \right\rfloor$ ;
3:   if ( $d == \text{vertical}$ ) then {
4:      $k.\text{value} = Y[m]$ ;
5:     PartitionField( $X, l, r, m$ );
6:   } else {
7:      $k.\text{value} = X[m]$ ;
8:     PartitionField( $Y, l, r, m$ );
9:   }
10:  ConstructBalanced2dTree( $l, m-1, k.\text{left}, !d$ );
11:  ConstructBalanced2dTree( $m+1, r, k.\text{right}, !d$ );
12: }
```

Call: ConstructBalanced2DTree($1, n, \text{root}, \text{vertical}$)

Run time

■ Construction

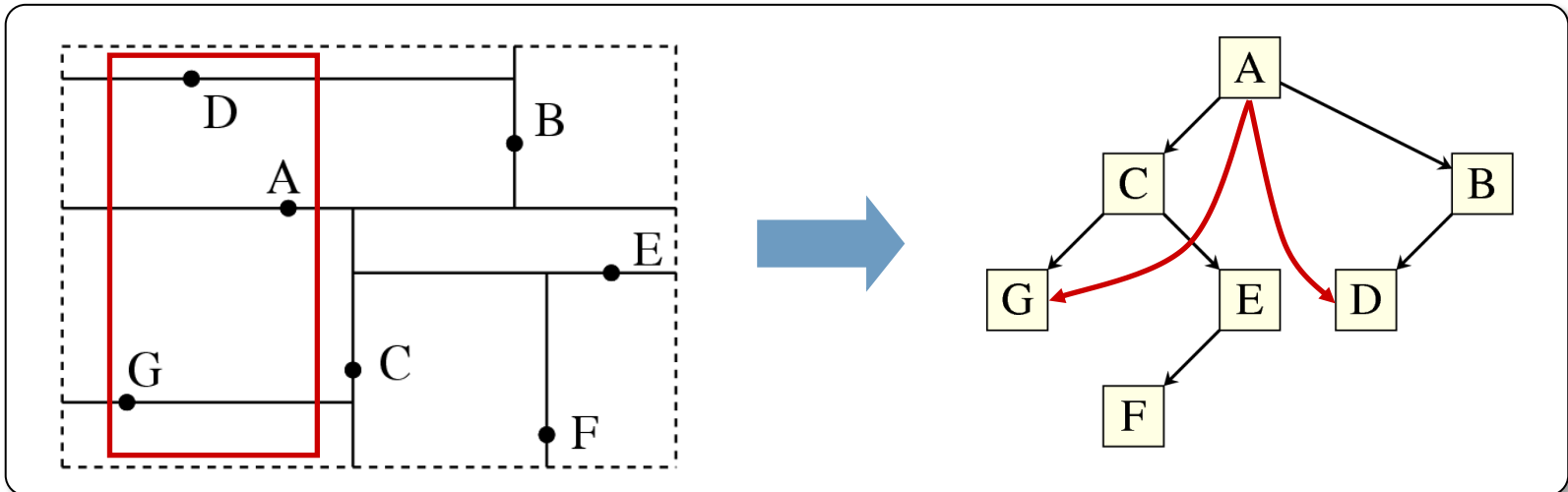
- Sorting of sequences: $O(n \log n)$.
- Partitioning of the sequences: $O(n)$.
- Recursive calls: $T(n) = T\left(\left\lceil \frac{n-1}{2} \right\rceil\right) + T\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) + O(n)$
- Solution of the recursion: $O(n \log n)$.

- Memory: $O(n)$, because a *kd*-tree is a binary tree with n leaves.

2c. kd-trees

Search in a 2d-tree

- The search is analogous to the usual search in a binary search tree.
- **Attention:** For every visited knot in the tree, the sub-trees of the knot are determined by the x -coordinate **or** the y -coordinate.
- **Attention*:** For a range search both sub-trees must be considered.
- **Example**



2c. kd-trees*

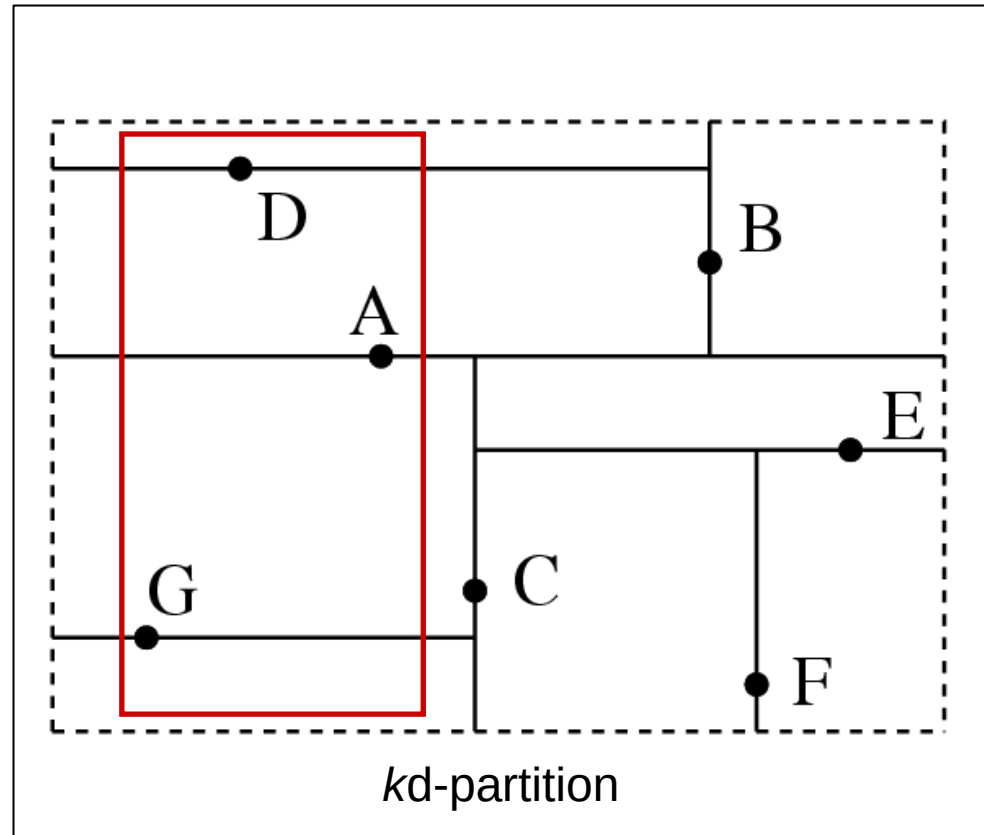
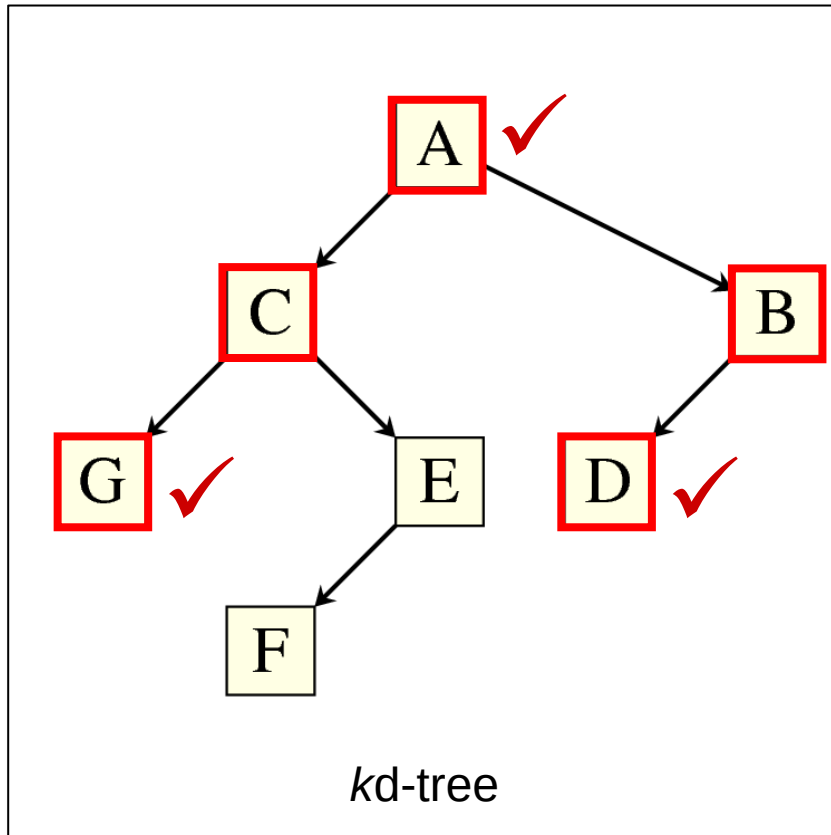
RangeSearch(knot k , direction d , query range D)

```
1: if ( $k \neq \text{NULL}$ ) then {
2:   if ( $d == \text{vertical}$ ) then {
3:     ( $l, r$ ) = ( $D.y_1, D.y_2$ );
4:     coord =  $k.y$ ;
5:     dNew = horizontal;
6:   } else {
7:     ( $l, r$ ) = ( $D.x_1, D.x_2$ );
8:     coord =  $k.x$ ;
9:     dNew = vertical;
10:  }
11:  if ( $k \in D$ ) then Add  $k$  to the output;
12:  if ( $l < \text{coord}$ ) then RangeSearch( $k.\text{left}$ , dNew,  $D$ );
13:  if ( $r > \text{coord}$ ) then RangeSearch( $k.\text{right}$ , dNew,  $D$ );
14: }
```

Call: RangeSearch(root , vertical , D)

2c. kd-trees

■ Example



Run time*

- Range search* in a balanced 2d-tree
 - Run time for R points in the query range D : $O(\sqrt{n} + R)$.
 - Proof: ... tricky!
 - For small R this is much faster than testing all points.

Higher dimensions

- Use the same algorithm!
- **For the construction** of the tree cycle through the k dimensions and partition the coordinate-sorted lists one by one.
- **For the search** also cycle through the k dimensions one after the other.
- **Run time and memory** in k dimensions
 - Construction: $O(k \cdot n \log n)$
 - Memory: $O(n)$ (A kd -tree is a binary tree with n leaves.)
 - Range search*: $O(kn^{1-\frac{1}{k}} + R)$

Content

- 1. Overview ✓
 - a. Intro: The 3d reconstruction pipeline ✓
 - b. Topology vs. geometry ✓
- 2. Point cloud data structures
 - a. Grids ✓
 - b. Quadtrees, octrees ✓
 - c. kd-trees ✓
 - d. **BSP-trees**
- 3. Mesh data structures
 - a. Mesh basics
 - b. Half-edge data structure

2d. BSP-trees

BSP-tree approach

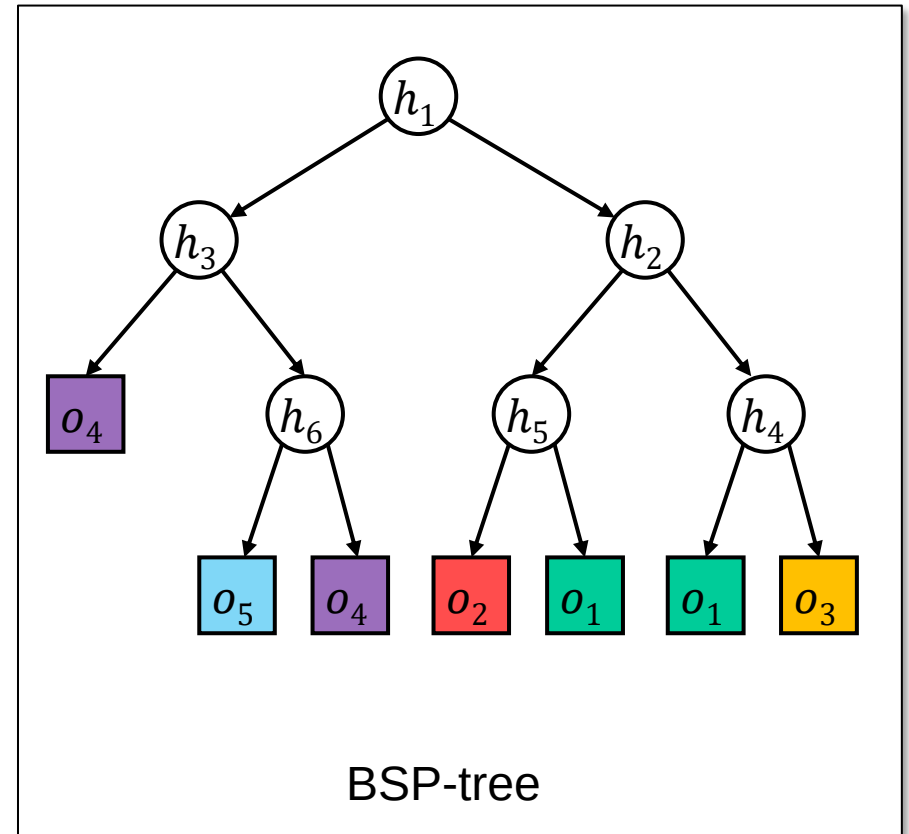
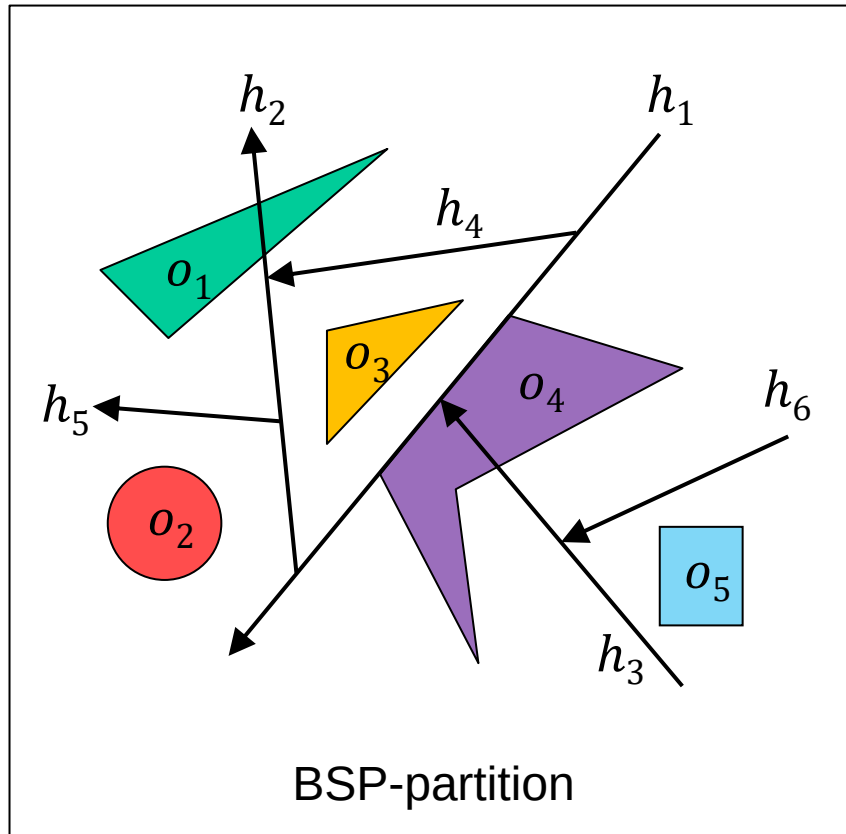
- Partition the point set P along arbitrary (hyper-)planes.
- **Efficient**, if the (hyper-)planes are axis-aligned, aka *two-dimensional trees (kd-trees)*.
- **Efficient**, if the (hyper-)planes are chosen arbitrarily aligned but optimal to yield a minimal depth.
- **Inefficient**, because the intersection of a rectangular query range with arbitrary half-spaces is an arbitrary convex polygon (convex polytope in kd).

2d. BSP-trees

- A data structure to sort and split polygons/polyhedrons and point clouds is a *Binary Space-Partitioning Tree* (*bsp-tree*).
- A BSP-tree partitions $\mathbb{R}^d$ recursively along arbitrary hyper-planes ($(d - 1)$ -dimensional affine sub-spaces).
 - Every inner node has two children: left or right of the hyper-plane.
 - Every inner node contains a splitting hyper-plane and possibly the $(d - 1)$ -dimensional objects contained in that hyper-plane.
 - The leaves partition $\mathbb{R}^d$ into the so-called *bsp-partition*.
 - The leaves contain the faces of the bsp-partition and the object fragments within this face.

2d. BSP-trees

Example:



2d. BSP-trees

- The hyper-plane h of an inner node is determined by a point $p_h \in \mathbb{R}^d$ and a (normalized) normal vector $n_h \in \mathbb{R}^d$ partitioning the space into two half-spaces

$$h^+ := \{q \mid (q - p_h)^t \cdot n_h > 0\},$$

$$h := \{q \mid (q - p_h)^t \cdot n_h = 0\},$$

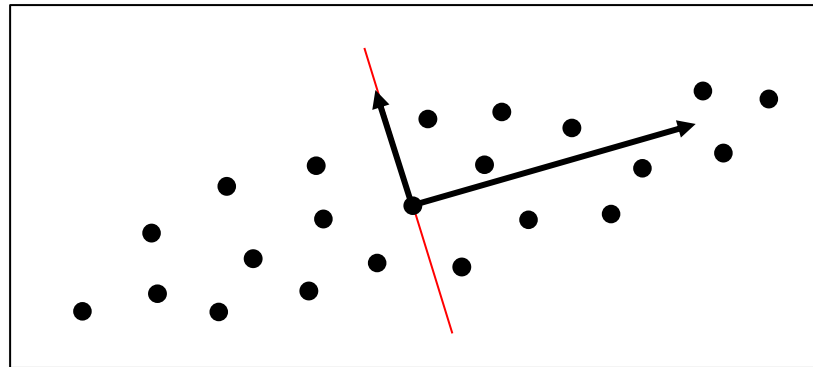
$$h^- := \{q \mid (q - p_h)^t \cdot n_h < 0\}.$$

- During the recursive construction of a bsp-tree, the objects are partitioned into the two half-spaces corresponding to the two child nodes.
 - Objects belonging to both half-spaces are split into fragments.
 - Objects completely contained in h are stored in a list corresponding to the node.

2d. BSP-trees

How to choose the best hyper-planes?

- If the objects stored are polygons/ polytopes/ polyhedrons/ free-form shapes, the hyper-planes can be defined by edges or faces of individual objects, yielding the so-called *auto-partitioning* strategy.
 - In 2d only edges (line segments) of objects need to be partitioned.
- If the object is a point cloud, the *principal component analysis (PCA)* yields a partitioning strategy.



2d. BSP-trees

Proposition

With a bsp-tree of size m the correct sorting of polygons along a given search direction z can be computed in $O(m)$.

Proof

- The sorting is generated traversing the sub-trees in the order

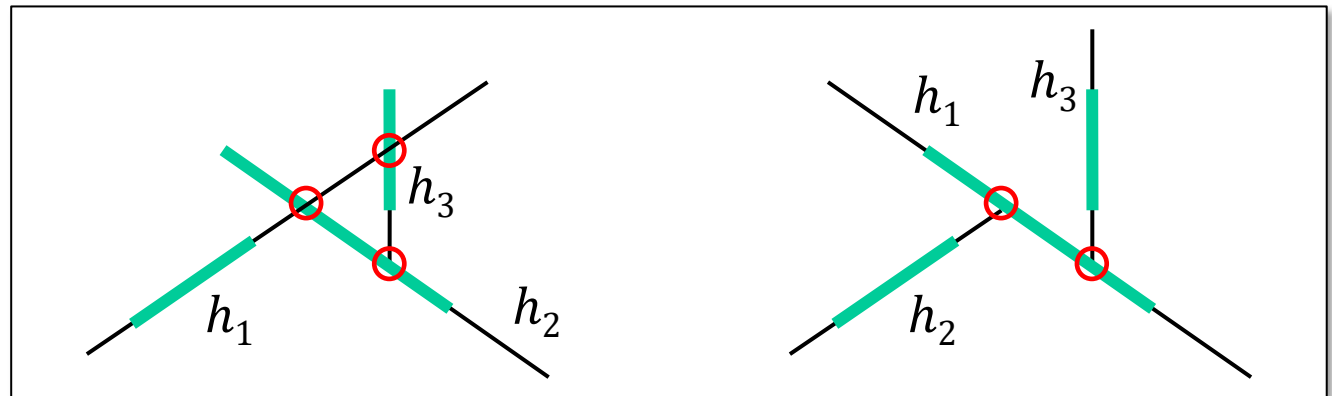
$$\begin{cases} h^+, h, h^-, & \text{if } z \cdot n_h > 0 \\ h^-, h, h^+, & \text{otherwise} \end{cases}.$$

- ▶ Every node is visited only once.
- Because of the splits, there are no cyclic overlaps.



2d. BSP-trees*

- Every inner node corresponds to a region of $\mathbb{R}^d$, defined by the intersection of all half-spaces of nodes above.
- The size of a bsp-tree, i.e., the number of its nodes, depends linearly on
 - the number of objects and
 - the number of fragments of objects, which depends on the sequence of splits.
- Example:



Content

- 1. Overview ✓
 - a. Intro: The 3d reconstruction pipeline ✓
 - b. Topology vs. geometry ✓
- 2. Point cloud data structures ✓
 - a. Grids ✓
 - b. Quadtrees, octrees ✓
 - c. kd-trees ✓
 - d. BSP-trees ✓
- 3. Mesh data structures
 - a. Mesh basics
 - b. Half-edge data structure

3a. Mesh basics

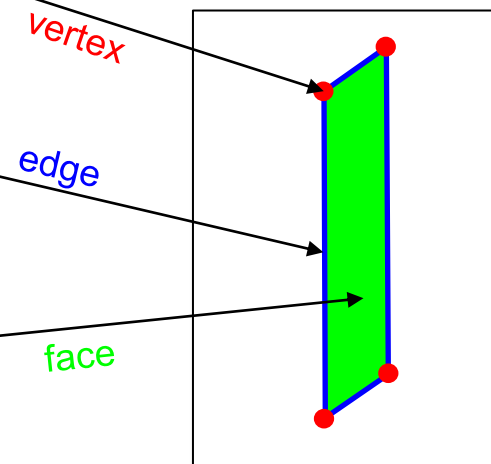
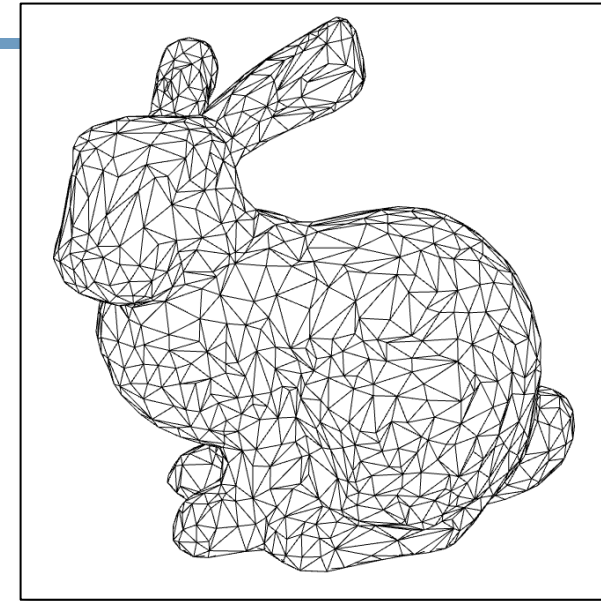
In **mesh-based modeling** objects are represented as meshes of points.

- **Advantage:** Adjacency is coded explicitly in the mesh.
- Such a mesh $\mathcal{M} = (\mathcal{V}, \mathcal{E}, \mathcal{F})$ consists of

- a set $\mathcal{V}$ of $n_{\mathcal{V}}$ **vertices**, which are associated with point information (coordinates, attributes, etc.),

- a set $\mathcal{E}$ of $n_{\mathcal{E}}$ **edges**, connecting two vertices,

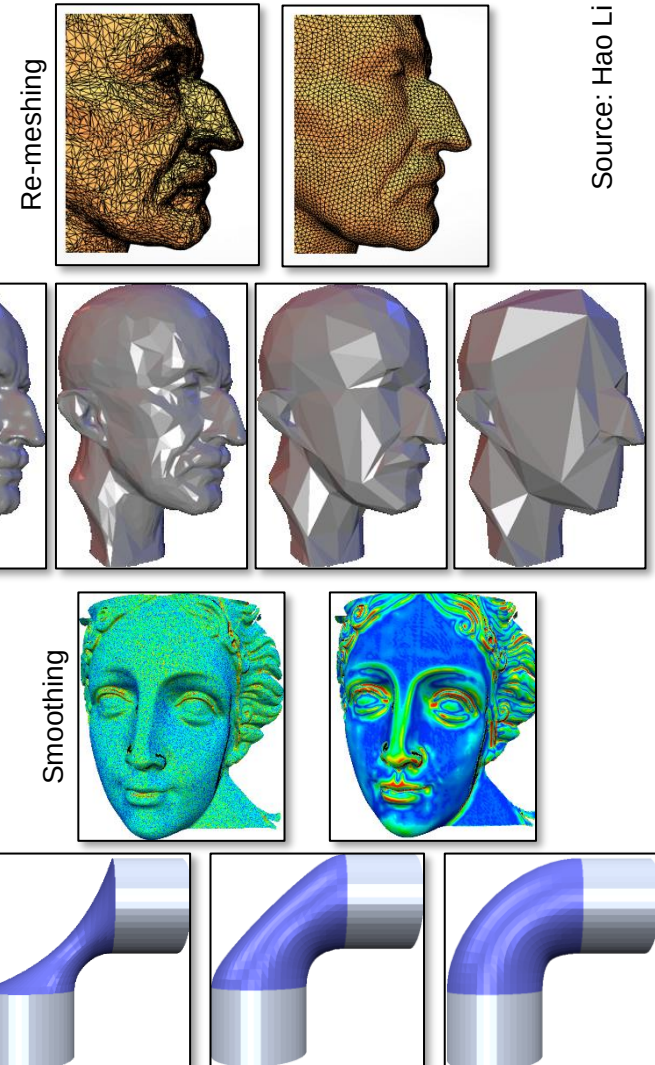
- a set $\mathcal{F}$ of $n_{\mathcal{F}}$ **faces**, i.e., a cyclic chain of edges without vertices in its interior.



3a. Mesh basics

Mesh-based processing objectives...

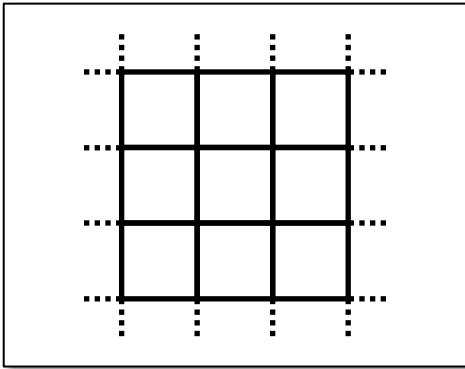
- *Re-meshing*: local/global operation
 - ▶ Improve shape of triangles and quality of mesh.
- *Decimation*: global operation
 - ▶ Reduce complexity of mesh
- *Smoothing*: local operation
 - ▶ Remove noise or errors
- *Fairing*: global operation
 - ▶ Simplify shape of mesh to yield a „simplest“ shape.
- *Rendering, Parametrization, Texturing, ...*
- *Deformation, Manipulation, Simulation, ...*
- *Boolean operations, etc.*



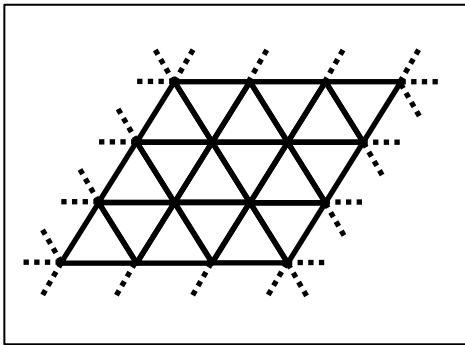
3a. Mesh basics

Mesh types (regular)

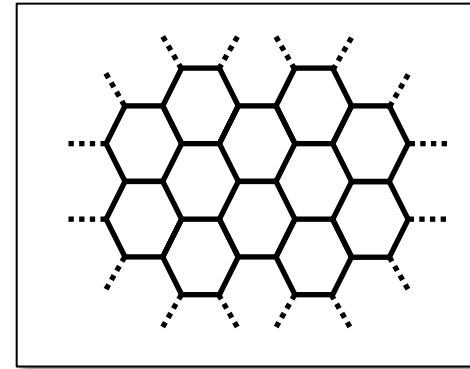
1. Regular quadrangular mesh



2. Regular triangular mesh



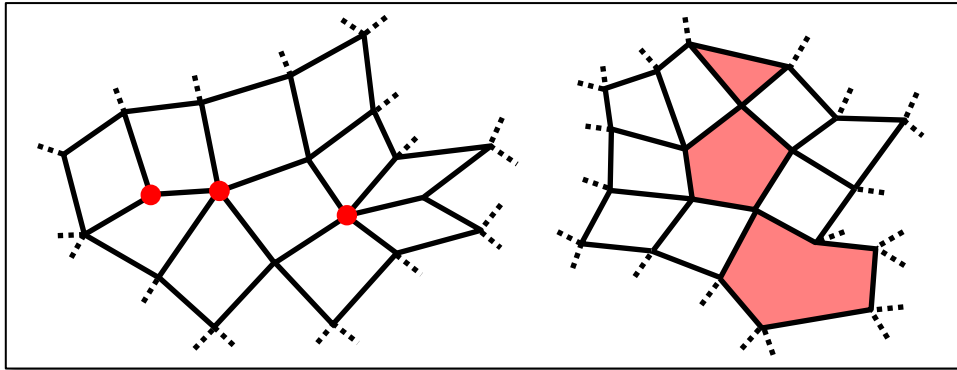
3. Regular hexangular mesh



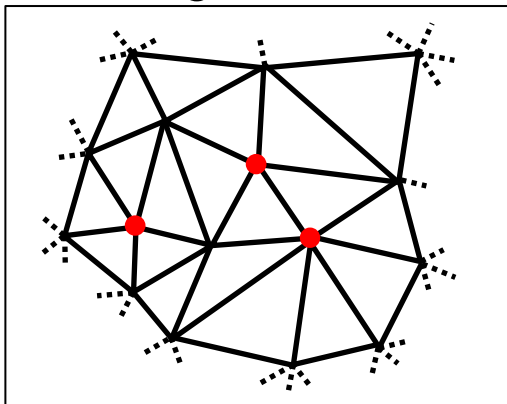
3a. Mesh basics

Mesh types (irregular)

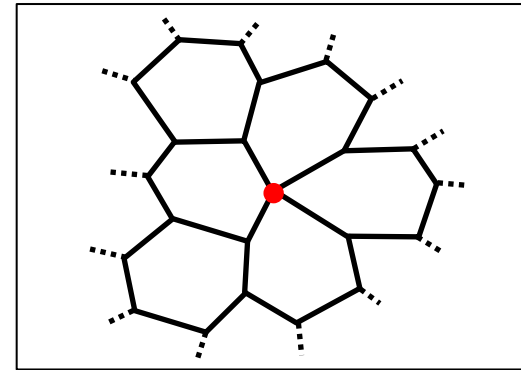
4. Irregular quadrangular mesh



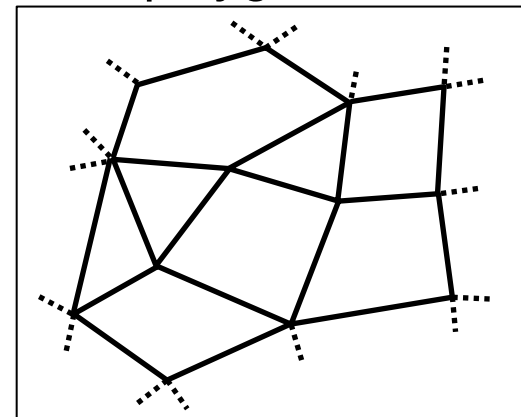
5. Irregular triangular mesh



6. Irregular hexangular mesh



7. General polygonal mesh



3a. Mesh basics

Local neighborhood

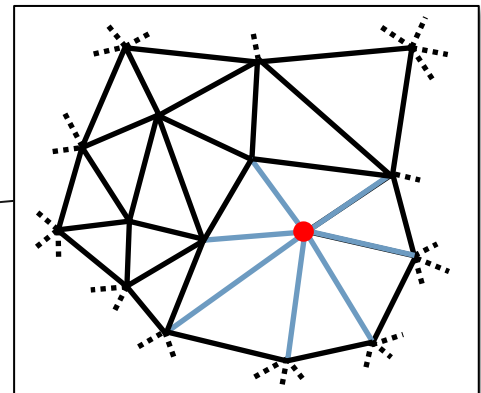
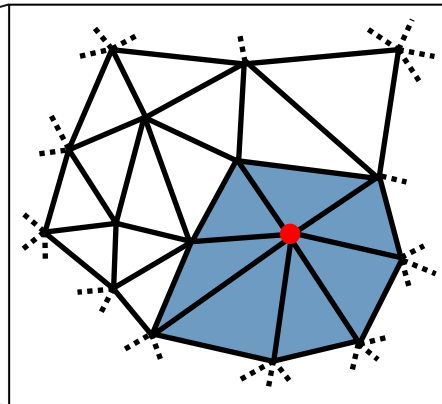
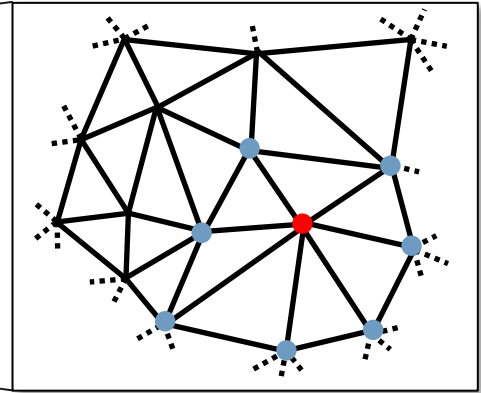
- **1-ring $\mathcal{N}_1(v)$ (aka 1-neighborhood):** All edge-adjacent neighbor vertices of a vertex v .

- ▶ **Valence:** Number of vertices in 1-ring.

- ▶ Define recursively 2-ring, 3-ring, etc.

- **1-fan $\mathcal{F}_1(v)$:** All faces immediately adjacent to a vertex v .

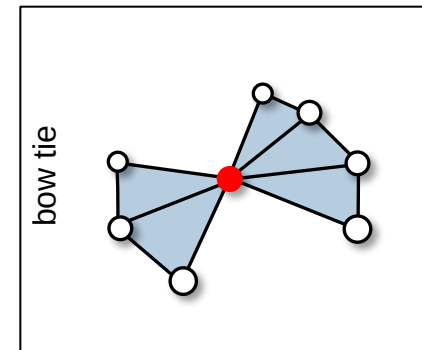
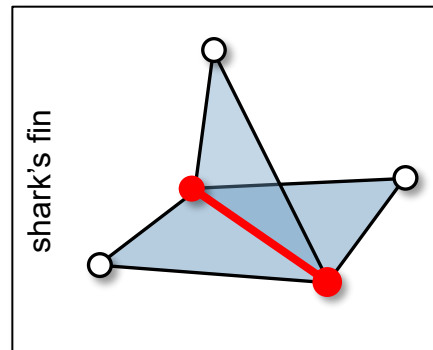
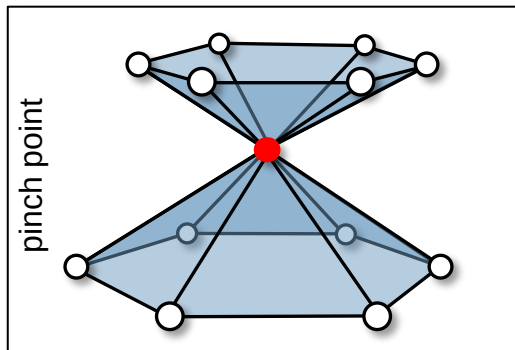
- **1-star $\mathcal{S}_1(v)$:** All edges emanating from a vertex v .



3a. Mesh basics

Topology (1)

- **2-manifold (aka surface):** Every vertex has a 1-fan, which is **homeomorphic** to a disc.
 - Homeomorphic: There exists a continuous, bijective mapping.
 - This is a notion of *local topology*.
- 2-manifold with boundary: ... homeomorphic to a half-disc.
- ▶ There are no **singularities**, as e.g.

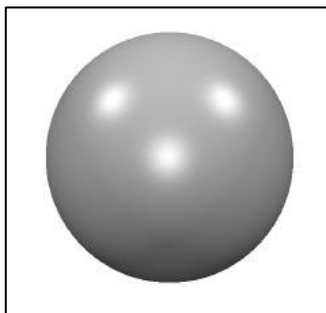


- 3-manifold (aka volume, solid), etc. analog

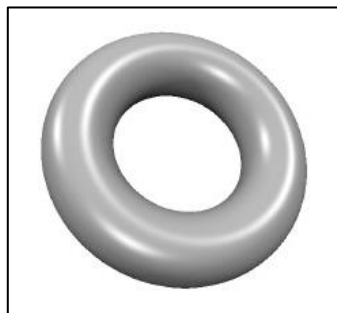
3a. Mesh basics

Topology (2)

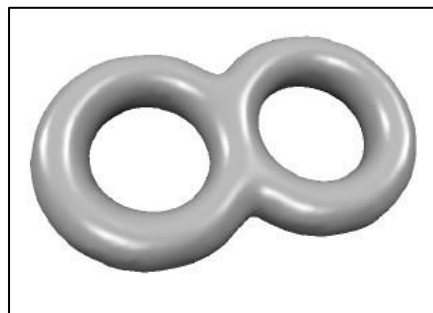
- The *global topology* of every manifold is determined by a small number of characteristic numbers (nof=number of):
 - Surface: nof components c + nof holes g + nof boundaries b
 - Solid: nof components + nof tunnels + nof bubbles + nof boundaries
- 2-manifold meshes (Euler): $n_V - n_E + n_F = 2(c - g) - b =: \chi$
 - g = genus (max. number of cuts while preserving connectedness)
 - χ = Euler characteristic
 - This is the connection between local and global topology for meshes.



genus 0



genus 1



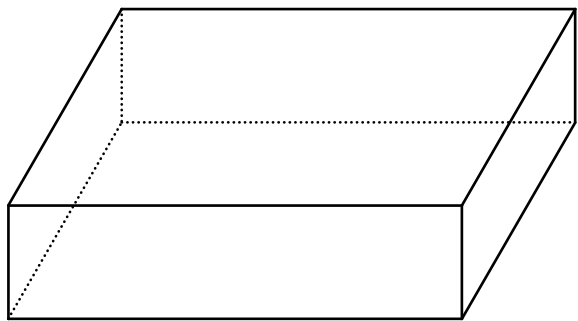
genus 2



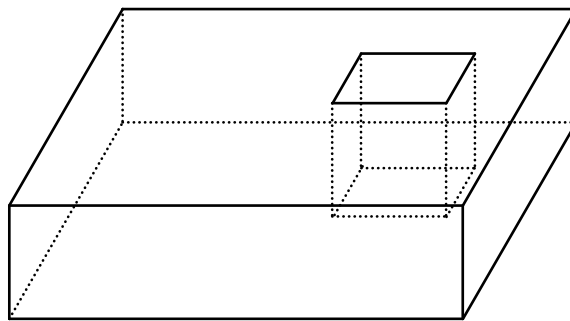
genus 3

3a. Mesh basics

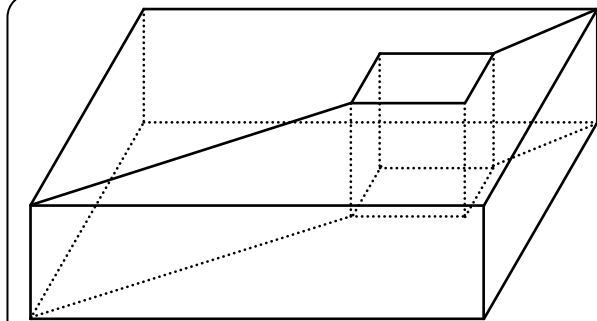
Examples: Topology of 3d meshes (1)



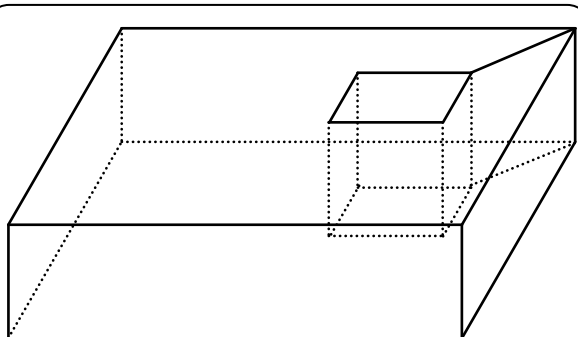
- $n_V = 8; n_E = 12; n_F = 6$
- $c = 1; g = 0; b = 0$
- $8 - 12 + 6 = 2(1 - 0) - 0 = 2$



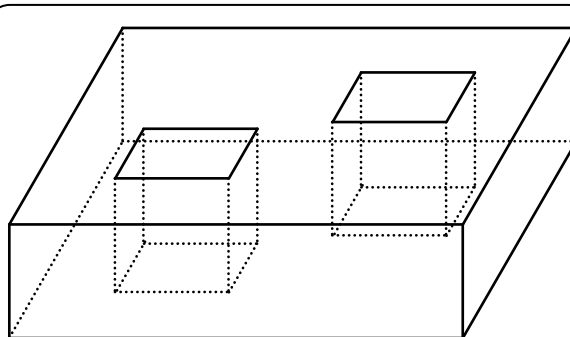
- $n_V = 16; n_E = 24; n_F = 12$
- $c = 2; g = 0; b = 0$
- $16 - 24 + 12 = 2(2 - 0) - 0 = 4$



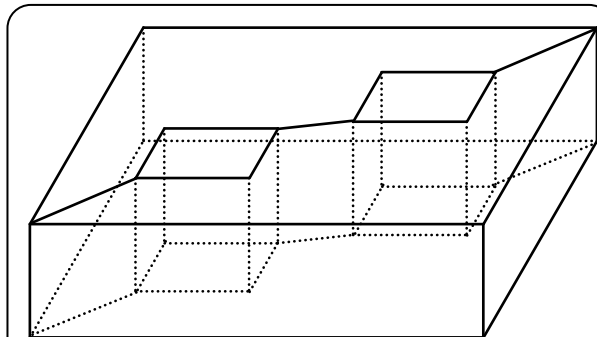
- $n_V = 16; n_E = 28; n_F = 12$
- $c = 1; g = 1; b = 0$
- $16 - 28 + 12 = 2(1 - 1) - 0 = 0$



- $n_V = 16; n_E = 26; n_F = 10$
- $c = 1; g = 1; b = 0$
- $16 - 26 + 10 = 2(1 - 1) - 0 = 0$



- $n_V = 24; n_E = 36; n_F = 18$
- $c = 3; g = 0; b = 0$
- $24 - 36 + 18 = 2(3 - 0) - 0 = 6$

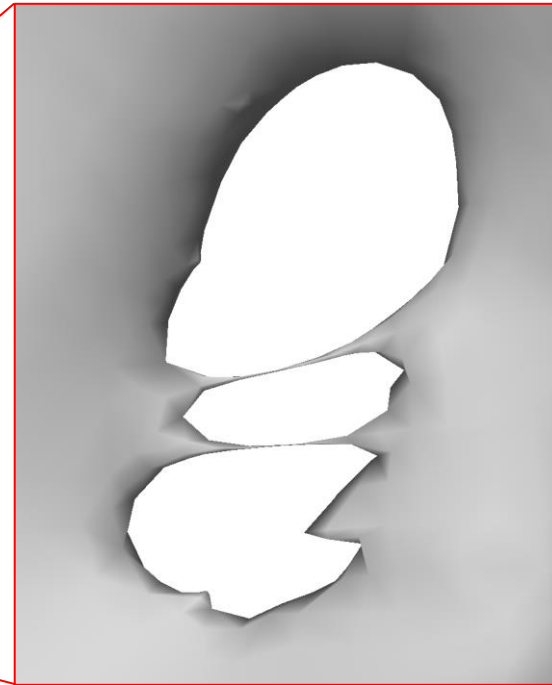
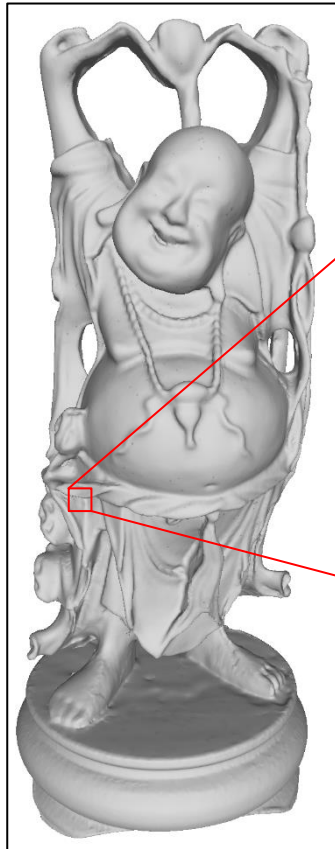


- $n_V = 24; n_E = 42; n_F = 16$
- $c = 1; g = 2; b = 0$
- $24 - 42 + 16 = 2(1 - 2) - 0 = -2$

3a. Mesh basics

Examples: Topology of 3d meshes (2)

Stanford Happy Buddha

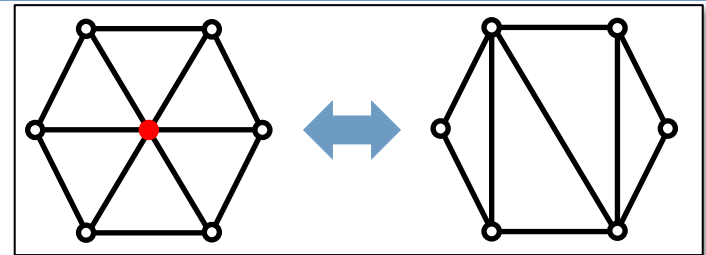


- $n_V = 543.652; n_E = 1.631.574; n_F = 1.087.716$
- $c = 1; g = ?; b = 0$
- $n_V - n_E + n_F = -206 = 2(1 - g) - 0 \Rightarrow g = 104$

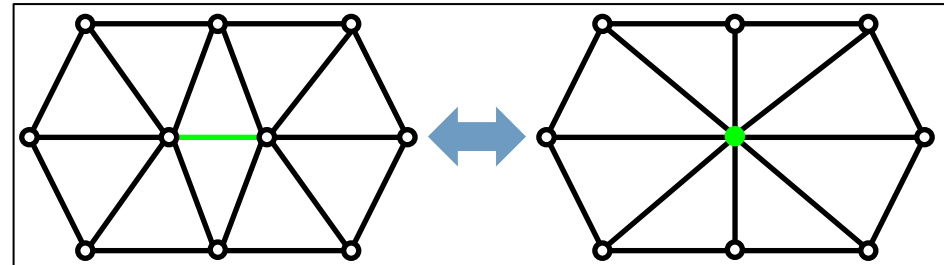
3a. Mesh basics

Topological operators

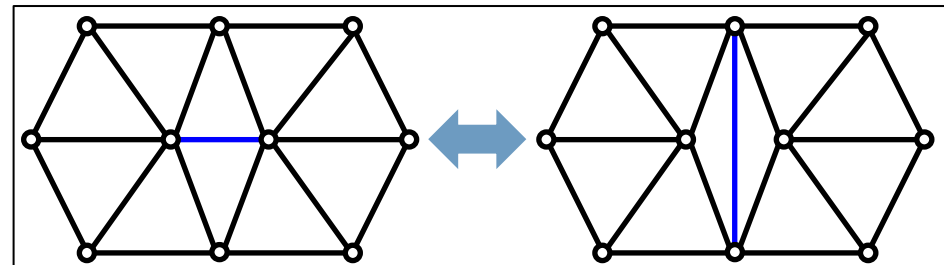
- **Vertex**-removal vs. **vertex**-insertion



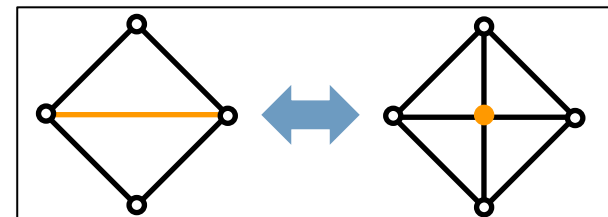
- **Edge**-collapse vs. **vertex**-split



- **Edge**-flip vs. **edge**-flip

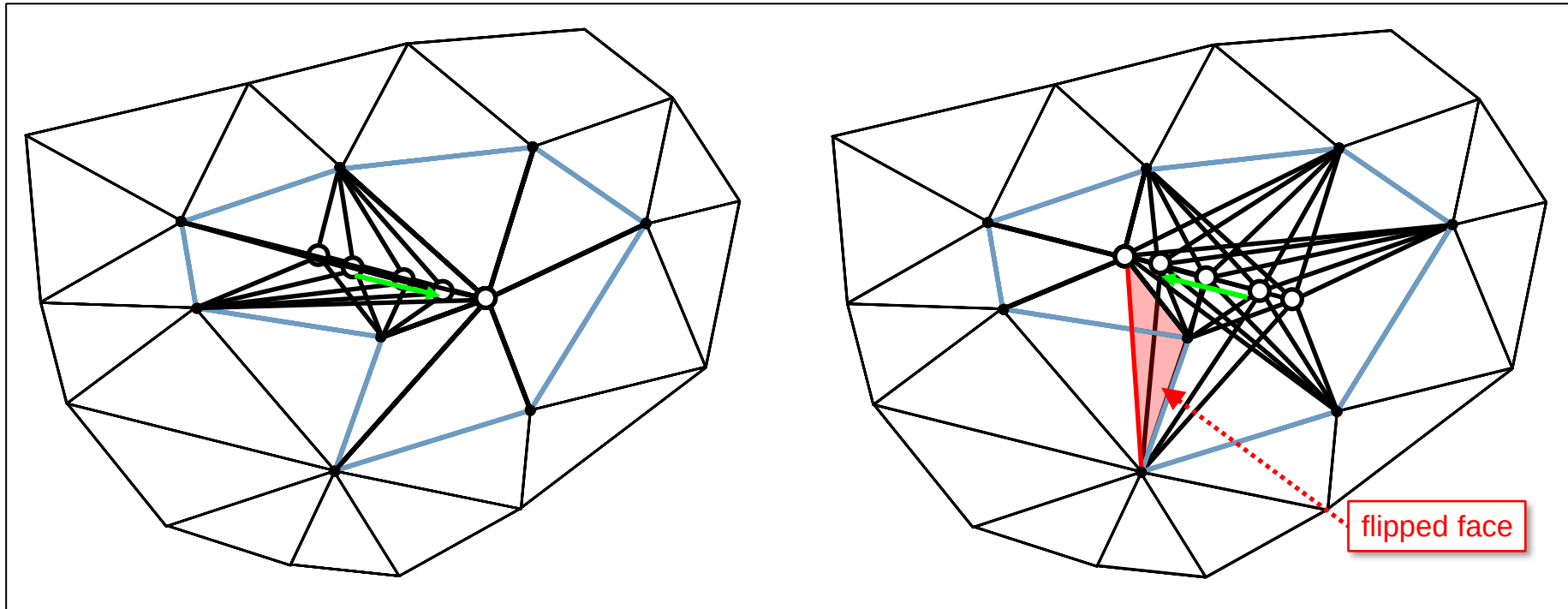


- **Edge**-split (vs. **vertex**-removal)



3a. Mesh basics

Coupling of topology and geometry for an edge-collapse

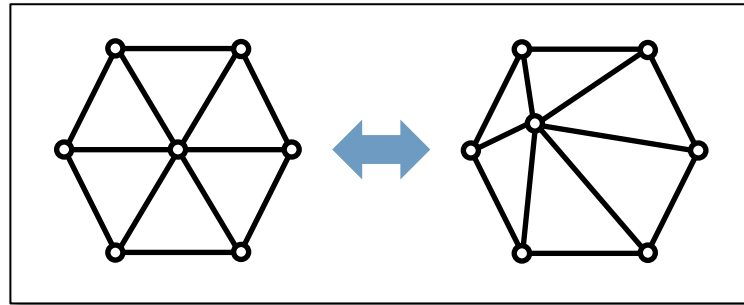


- ▶ For topological operations, check for geometric plausibility!
- ▶ Often topological and geometric operations are coupled.

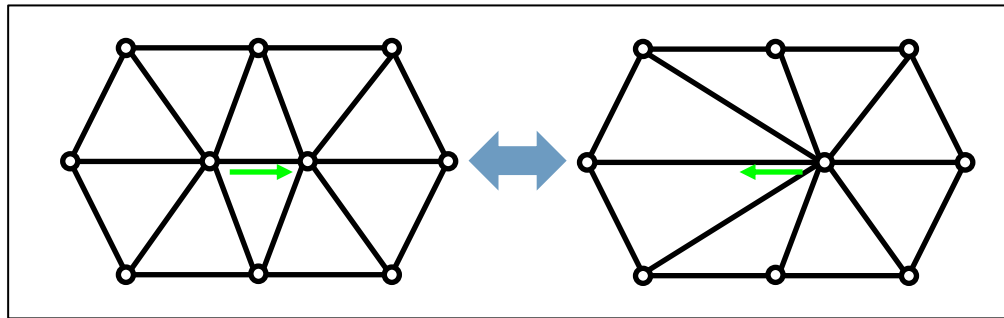
3a. Mesh basics

Geometrical operators

- Vertex-shift



- Half-edge-collapse vs. restricted-vertex-split (topological & geometrical)



3a. Mesh basics

Combinatorial operators

- Vertex-based
 - Find and enumerate all adjacent vertices, aka 1-ring.
 - Find and enumerate all edges emanating from a given vertex, aka 1-star.
 - Find and enumerate all adjacent faces, aka 1-fan.
- Edge-based
 - Find start- and end-vertex.
 - Find successor-/predecessor edge in adjacent face.
 - Find adjacent faces.
- Face-based
 - Find and enumerate all adjacent vertices, aka polygon corners.
 - Find and enumerate all bounding edges.
 - Find and enumerate all adjacent faces.

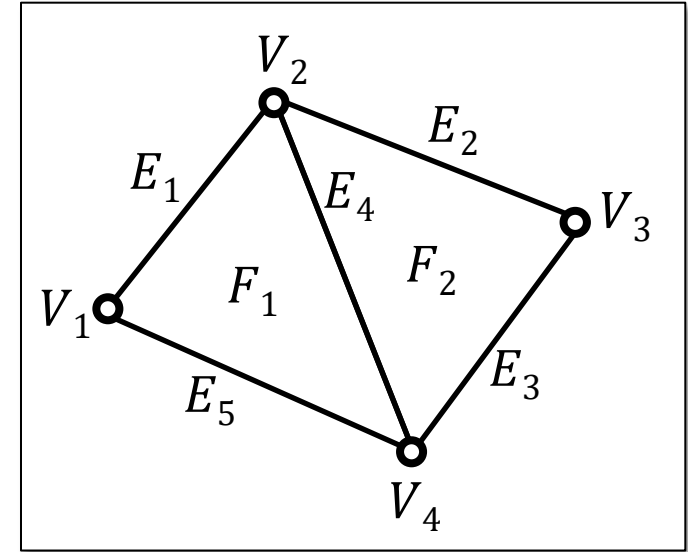
Content

- 1. Overview ✓
 - a. Intro: The 3d reconstruction pipeline ✓
 - b. Topology vs. geometry ✓
- 2. Point cloud data structures ✓
 - a. Grids ✓
 - b. Quadtrees, octrees ✓
 - c. kd-trees ✓
 - d. BSP-trees ✓
- 3. Mesh data structures
 - a. Mesh basics ✓
 - b. Half-edge data structure

3b. Half-edge data structure

A mesh is stored as

- winged-edge data structure¹,
- half-edge data structure¹,
- directed-edge¹,
- quad-edge¹,
- combinatorial-map data structure¹,
- radial-edge data structure² [Weiler88],
- feather data structure² [Yamaguchi/Kimura95],
- etc.



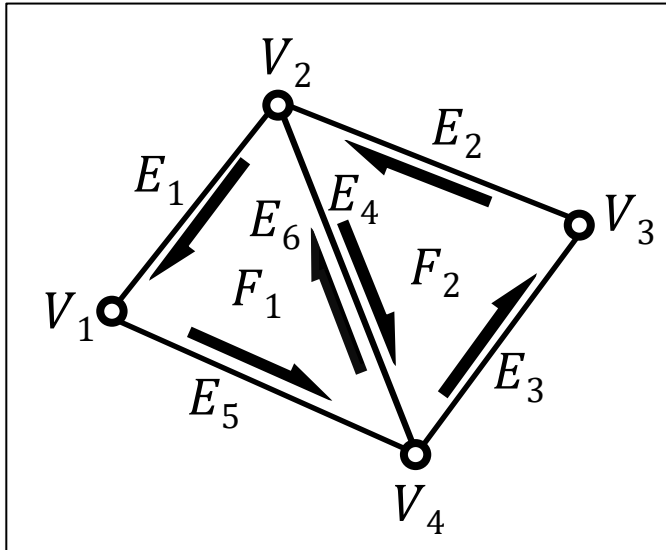
¹ only applicable for 2-manifolds

² also applicable to non-manifold meshes and volume-meshes

3b. Half-edge data structure

Half-edge data structure

- Each edge is split into a pair of half-edges.
- Pointers should be realized as indices to improve efficiency.



Vertex

```
Point      position;  
HalfEdge*  one_outgoing_half_edge;
```

HalfEdge

```
Vertex*    end_vertex;  
Face*      adjacent_face;  
HalfEdge*  next_half_edge_in_face;  
HalfEdge*  prev_half_edge_in_face;  
HalfEdge*  opposite_half_edge;
```

Face

```
HalfEdge*  one_of_its_half_edges;
```

- ▶ This allows for an efficient implementation of all topological, geometrical, and combinatorial operations.

3b. Half-edge data structure

Example operation

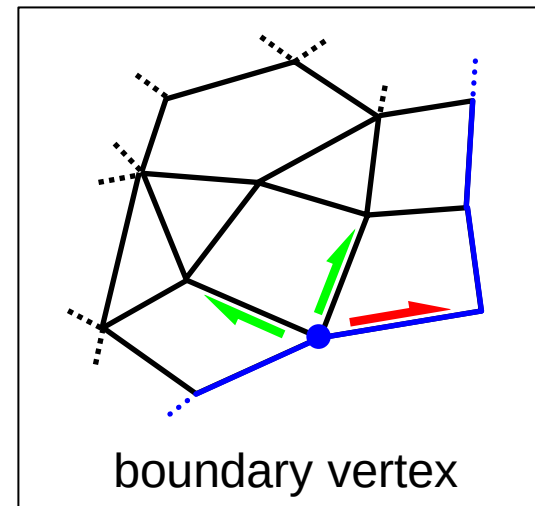
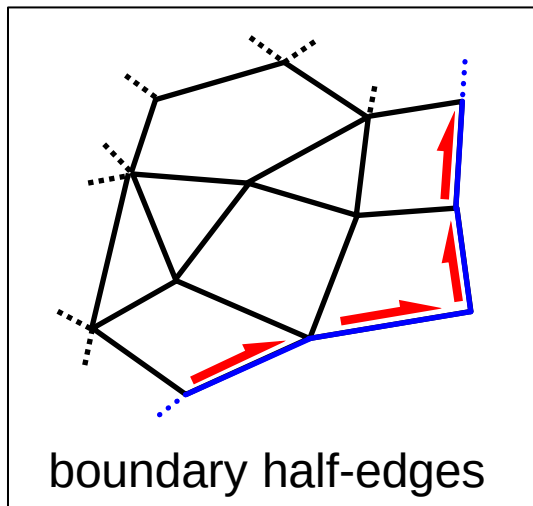
- Enumerate 1-ring for a vertex **vtx** and apply a function **func** to its vertices.

```
Enumerate_One_Ring(Vertex& vtx, Function func)
{
    HalfEdge* h      = vtx.one_outgoing_half_edge;
    HalfEdge* h_stop = h;
    do {
        Vertex* v = h->end_vertex;
        func(v); // process vertex v
        h = (h->prev_half_edge_in_face)->opposite_half_edge;
    } while (h!=null && h!=h_stop);
}
```

3b. Half-edge data structure

Half-edge data structure

- For meshes with boundary
 - the opposite-half-edge of a **half-edge** at the **boundary** is undefined, or **null** or negative or ...
...and...
 - the outgoing **half-edge** in a **vertex** at the **boundary** must be “**right-hinged**”.



3b. Half-edge data structure

Directed-edge data structure

- For **triangle-meshes** the half-edge data structure can be specialized using indices, following some rules.

- For a face with index f , the indices of the three half-edges are

$$\text{half_edge}(f) = 3f + i, \quad i = 0, 1, 2.$$

- For a half-edge with index h , the index of its face and index within this face are

$$\text{face}(h) = h/3,$$

$$\text{face_index}(h) = h \bmod 3.$$

- For a half-edge with index h , the index of its next half-edge in its face is

$$\text{next_half_edge}(h) = (h + 1) \bmod 3.$$

Vertex

```
Point position;  
EdgeId outoing_half_edge;
```

HalfEdge

```
VertexId end_vertex;  
EdgeId opposite_edge;
```

Face

```
// empty
```

► Memory consumption

- 16 bytes/vertex
- 8 bytes/half-edge

Content

- 1. Overview ✓
 - a. Intro: The 3d reconstruction pipeline ✓
 - b. Topology vs. geometry ✓
- 2. Point cloud data structures ✓
 - a. Grids ✓
 - b. Quadtrees, octrees ✓
 - c. kd-trees ✓
 - d. BSP-trees ✓
- 3. Mesh data structures ✓
 - a. Mesh basics ✓
 - b. Half-edge data structure ✓

Questions

- What is the difference between topology and geometry?
- What is an octree and what are its pros and cons?
- What is a *kd*-tree and do you implement the construction of a *kd*-tree efficiently?
- What is a bsp-tree and where do you get the splitting planes from?
- Where did we make use of efficient space-partitioning trees in the lecture?
- What is an edge-collapse?
- What are a half-edge and directed-edge data structures and what is the difference between the two?